Capítulo 23

Invocación remota

Al terminar este capítulo, entenderás:

- Qué es un protocolo de petición-respuesta.
- Cuáles son las distintas semánticas de invocación.
- Qué es y cómo funciona la invocación remota.
- Qué son RPC y RMI.
- Qué es la transparencia de localización.
- Qué alternativas plantean Cap'n Proto y REST.
- Cuánto cuesta realmente la invocación remota.

El capítulo 15 presentó el patrón petición-respuesta; el cliente envía una petición —normalmente una operación sobre un recurso— y el servidor responde con un resultado. Ese patrón lo define, en cada caso, un protocolo de aplicación propio: HTTP para la web, SMTP para el correo, DNS para la resolución de nombres, etc. Cada uno especifica su propio formato de mensajes, sus propias operaciones y su propia forma de indicar errores, adaptados al problema concreto que resuelven.

Diseñar un protocolo de aplicación desde cero no es trivial; hay que definir la sintaxis de los mensajes, decidir cómo se delimitan sobre el flujo de bytes subyacente, considerar la evolución del protocolo (versiones, compatibilidad) y, finalmente, implementar esa especificación en cada lenguaje o plataforma que vaya a usarla. Para muchas aplicaciones, ese coste es desproporcionado en relación con lo que realmente necesitan: simplemente ejecutar código en un nodo remoto y obtener el resultado, igual que se invoca una función o un método local.

La **invocación remota** es una solución general para esa necesidad. En lugar de definir un protocolo específico por aplicación, ofrece al programador una infraestructura genérica —el **middleware** de comunicaciones—, común a cualquier aplicación, que se encarga de transportar la invocación hasta el nodo remoto, ejecutarla allí y devolver el resultado. RPC (Remote Procedure Call) sigue este enfoque para procedimientos (funciones indepen-

dientes); RMI (Remote Method Invocation) hace lo mismo para métodos de objetos. En ambos casos, el middleware genera el código encargado de establecer la comunicación, serializar los argumentos, enviarlos, ejecutar el código, devolver la respuesta y entregar el resultado de vuelta al cliente, de modo que invocar una operación remota se parece, desde el punto de vista del programador, a invocar una operación local.

Lo que realmente diferencia una invocación remota de un mero intercambio de mensajes es que la invocación implica la ejecución de un fragmento de código en el receptor a expensas del emisor. La petición indica qué pieza de código se ha de ejecutar y con qué argumentos, mientras que la respuesta transporta el resultado.

En la invocación remota, el que envía el mensaje de petición (la invocación) es el cliente, y el que recibe y ejecuta el código asociado es el servidor. Sin embargo, aunque es lo habitual, esos roles no tienen por qué coincidir con los de los procesos involucrados, es decir, un servidor TCP podría ser el que envía la invocación hacia el cliente TCP.

23.1. Protocolos de petición-respuesta

Dos procesos que se ejecutan en nodos distintos no comparten memoria, así que la única forma de interactuar es enviándose mensajes a través de la red: eso es el *paso de mensajes*. Se reduce, en su forma más básica, a dos primitivas sobre un socket, `send(destino, mensaje)` y `receive(origen, mensaje)`, sin que deba existir ninguna relación entre mensajes sucesivos.

El patrón *petición-respuesta* es la forma más simple de paso de mensajes en una comunicación extremo a extremo: asocia un mensaje de respuesta al de petición que la originó. Este patrón lo emplean protocolos de varias capas, pero es especialmente frecuente en la de aplicación.

Diseñar un protocolo de aplicación obliga al diseñador a tomar esencialmente las mismas decisiones, independientemente del problema que resuelva el protocolo, aunque el resultado de esas decisiones varía según las necesidades de la aplicación.

La invocación remota no es diferente, aunque no resuelve todas esas decisiones del mismo modo: el middleware fija de antemano algunas de ellas al generar automáticamente el protocolo subyacente, mientras que deja otras en manos del programador. Estas son algunas:

- **Síncrono vs. asíncrono**: el cliente de la petición puede bloquear en espera de la respuesta, o continuar inmediatamente con otra tarea. La primera es una invocación síncrona y es más fácil de entender,

porque el cliente sabe en todo momento si la petición sigue pendiente; además es más similar a una llamada local. Una invocación asíncrona aprovecha el tiempo de espera, pero debe encargarse de correlacionar más adelante cada respuesta con la petición que le corresponda.

- **Con o sin estado**: el servidor puede mantener el estado entre peticiones sucesivas del mismo cliente, o resolver cada una de forma independiente, sin memoria de las anteriores. Un protocolo sin estado es más sencillo de escalar y de recuperar tras un fallo, porque una réplica del servicio puede atender la siguiente petición sin compartir contexto previo con la que atendió la anterior; a cambio, cualquier información que dependa de peticiones anteriores tiene que incluirse explícitamente en cada una.
- **Transporte y fiabilidad**: sobre TCP la conexión ya garantiza entrega y orden, pero establecerla tiene un coste que no siempre compensa para un intercambio breve. Sobre UDP no hay tal garantía: si se pierde el mensaje con la petición o el que lleva la respuesta, nada lo detecta ni corrige.

Entre la fiabilidad completa que ofrece TCP y la ausencia total de UDP hay un término medio (o varios): el protocolo de petición-respuesta puede construir sobre UDP solo las medidas de fiabilidad que necesite, sin la sobrecarga que implican las que no necesita. El identificador de mensaje, como el que llevan las consultas DNS, permite cotejar cada respuesta con la petición que la originó, aunque haya varias en curso o el transporte las reordene; ese mismo identificador le permite al servidor detectar y descartar duplicados en lugar de procesarlos de nuevo. Un temporizador permite detectar peticiones o respuestas perdidas. Con ese mismo temporizador se puede añadir la retransmisión, reintentando la petición un número limitado de veces antes de darla por fallida definitivamente. Identificador, temporizador y retransmisión son las piezas básicas de cualquier protocolo ARQ; cada protocolo de petición-respuesta elige cuáles de ellas necesita, y con qué grado de exigencia.

Estos elementos definen tres estilos de intercambio, según qué mensajes intervienen y con qué fin:

- R (Request): el cliente envía la petición y no espera ninguna respuesta (no bloqueante). Sirve para peticiones *one-way* habitualmente sobre transporte no confiable. No hay un valor de retorno, confirmación, temporizadores ni retransmisiones. También podría ocurrir que hubiera un resultado o confirmación, pero al cliente de la petición no le interese, por lo que no espera ni recoge el mensaje.

- RR (Request-Reply): El cliente espera un mensaje de respuesta (bloqueante). La respuesta, además de posiblemente contener un resultado, puede llevar un código de confirmación o de error. Puede utilizar TCP como transporte o bien implementar confiabilidad básica con un ARQ. RR es el estilo más común en protocolos de aplicación.
- RRA (Request-Reply-Acknowledge): cuando el servidor detecta una petición duplicada, no basta con descartarla: si ya ejecutó la operación, tiene que reenviar el mismo resultado en lugar de ejecutarla de nuevo, así que necesita conservarlo en una caché llamada *historia*, indexada por el identificador de mensaje, en lugar de descartarlo justo después de enviarlo. La duda es cuánto tiempo mantener cada entrada: sin ninguna señal del cliente, el servidor solo puede adivinarlo con un temporizador propio y un margen de seguridad. Este estilo añade un tercer mensaje con el que el cliente confirma la recepción de la respuesta, y con el que el servidor sabe explícitamente cuándo puede liberar la entrada de la historia. Sin este mensaje, la historia crecería indefinidamente.

23.2. Medidas de tolerancia a fallos

Frente a la pérdida de un mensaje, la invocación remota puede aplicar tres medidas, que ofrecen fiabilidad creciente:

- **Reintentar la petición**: si el temporizador expira sin recibir respuesta, el cliente retransmite el mismo mensaje, con el mismo identificador. Se aplica un número limitado de repeticiones, que de alcanzarse, será considerado un error. Obviamente implica obligatoriamente un mensaje de respuesta/confirmación. Sin esta medida, el cliente no puede saber si la petición llegó al servidor.
- **Filtrar duplicados**[1]: el servidor conserva los identificadores de las peticiones ya atendidas, para reconocer una retransmisión y no repetir la ejecución de la operación que le corresponde. Sin este filtro, cada retransmisión se procesa de nuevo.
- **Retransmisión del resultado**: al detectar una petición duplicada, el servidor no se limita a descartarla, sino que reenvía el resultado que ya calculó y tiene guardado, en lugar de dejar al cliente sin respuesta pese a que la operación sí llegó a ejecutarse.

[1] denominado a menudo «deduplicación»

23.3. Operaciones idempotentes

Una operación es **idempotente** cuando ejecutarla varias veces (con los mismos argumentos) produce sobre el estado del servidor el mismo efecto que ejecutarla una sola vez. La idempotencia es una propiedad de la semántica propia de la operación. Es interesante porque estas operaciones no requieren detección de duplicados, lo que simplifica su implementación.

Un estilo RRA necesita cachear la respuesta de cada petición precisamente en el caso de las operaciones no idempotentes, que normalmente se asume que es caso más común.

Algunos ejemplos de operaciones idempotentes podrían ser:

```
set_size(25)
stop()
car.get_color()
get_time()
```

Es interesante destacar que `get_time()()` sea idempotente, porque aunque el valor que retorne sea diferente en cada llamada, cumple la condición de no modificar el estado del servidor.

Mientras que los siguientes son no idempotentes:

```
push("item")
counter.inc()
send_mail("user@example.com", "Hello")
```

Este concepto de idempotencia no coincide con el que se utiliza en matemáticas. Aquel está relacionado con el resultado de la aplicación de una función al resultado previo de esa misma función, que de hecho ni siquiera aplica a varios de los ejemplos anteriores porque requiere que la función tenga al menos un argumento.

23.4. Semántica de invocación

Combinar las tres medidas de tolerancia a fallos —reintentar petición, filtrar duplicados y retransmitir resultado— da lugar a distintas garantías sobre potencialmente cuántas veces se llega a ejecutar la operación remota. Estas garantías se conocen como semánticas de invocación:

- *maybe*: no aplica ninguna medida. Si se pierde la petición o la respuesta, el cliente no tiene forma de saberlo ni de corregirlo. La operación se ejecuta cero o una vez, sin que el cliente pueda distinguir cuál de los dos casos ha ocurrido. Es la semántica que resulta, por ejemplo, de un estilo R (Request) sobre un transporte no confiable. Por ejemplo

es útil cuando el cliente envía mensajes actualizando periódicamente el valor de una medida (*p. ej.* la temperatura de una habitación), y no es un problema que se pierda alguno de esos mensajes.

- *at-least-once*: reintenta la petición hasta tener una confirmación, pero no filtra duplicados. Cada retransmisión que llega al servidor se ejecuta de nuevo, así que la operación puede llegar a ejecutarse una o más veces. Garantiza que se ejecuta al menos una vez, a costa de no poder garantizar que no se repita. Se puede implementar con el estilo RR o R con transporte confiable si no requiere valor de retorno.

- *at-most-once*: además de reintentar, el servidor filtra los duplicados. Conserva el resultado ya calculado de la primera ejecución para volver a enviarlo si la petición se repite. La operación se ejecuta como mucho una vez (0 o 1 veces). El cliente siempre recibe una respuesta, ya sea para informar del resultado o de un error, es decir, a diferencia de *maybe* el cliente sabe si la operación se ejecutó o no. Es la semántica que ofrece el estilo RRA (Request-Reply-Acknowledge).

- *exactly-once*: ejecuta la operación exactamente una vez. Esto es algo que las tres medidas de tolerancia a fallos no pueden garantizar por sí solas, porque únicamente actúan sobre la comunicación. La garantía *exactly-once* requiere asegurar la continuidad del estado del servidor, particularmente entre la ejecución de la operación y su registro en la historia, por lo que ambas se ejecutan como una operación atómica. Además requiere de un sistema adicional para recuperar al proceso servidor tras un *crash* con recuperación transparente del estado previo. Eso lo convierte en un problema realmente complejo y difícilmente justificable. En la práctica, lo que se suele implementar es una variante más simple llamada *effectively-once*, que es una combinación de *at-least-once* sobre operaciones idempotentes.

El fabricante de cada middleware decide qué semánticas de invocación ofrece, en función de los estilos de intercambio y de las medidas de tolerancia a fallos que implementa. Al programador le corresponde elegir, entre las disponibles, la más adecuada en cada caso, y tomar otras decisiones que el middleware no impone, como invocar de forma síncrona o asíncrona. Esto es una ventaja, le otorga mayor control para adaptar cada invocación a las necesidades y requisitos de la aplicación.

23.5. Mecanismo de invocación remota

Con independencia de la tecnología concreta —RPC, RMI u otra—, todo mecanismo de invocación remota comparte una arquitectura similar. Comienza con una *interfaz remota* conocida tanto por el cliente como por el servidor. La interfaz es una lista de las operaciones que ofrecerá (implementará) el servidor, con sus nombres, parámetros y tipos de retorno, mientras que el cliente será quien invoque esas operaciones.

El middleware proporciona habitualmente un compilador que genera el código necesario para transportar las invocaciones (identificadores y argumentos) hasta el servidor y llevar los resultados de vuelta al cliente.

Esos fragmentos de código se denominan *stubs* (cabos) y realizan tareas diferentes para cliente y el servidor. Los *stubs* del cliente proporcionan funciones que implementan la interfaz remota, pero no tienen funcionalidad de la aplicación. Lo que hacen es serializar y transportar las invocaciones hasta el servidor. Los *stubs* del servidor incluyen la lógica de multiplexación que localiza y ejecuta funciones con las mismas firmas. Es habitual que ese compilador ofrezca la posibilidad de generar implementaciones huecas para las operaciones, que suelen denominarse *esqueletos (skeletons)*.

La Figura 23.1 resume el proceso completo. El compilador de interfaces genera los *stubs* de cliente y de servidor a partir de la interfaz, y el compilador de cada lenguaje los combina con el código que escribe el programador para obtener los ejecutables. En la figura, el cliente está escrito en Java y el servidor en C++, de modo que el compilador de interfaces genera el *stub* de cliente en Java y el de servidor en C++.

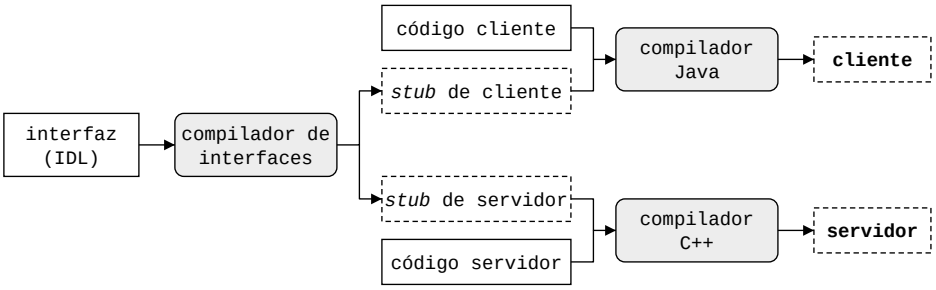


FIGURA 23.1: Generación de *stubs* y ejecutables a partir de la interfaz (en discontinua los ficheros generados)

El recorrido completo de una invocación remota, desde que el cliente invoca su *stub* hasta que recibe el resultado, sigue siempre los pasos del Cuadro 23.1.

Cliente	**Servidor**
Invoca la operación remota pasando los argumentos correspondientes, de forma similar a como invocaría una función o un método local. En realidad está invocando al *stub*.	
El *stub* serializa el identificador de la operación junto con los argumentos , y con eso construye el mensaje de petición.	
El *stub* del cliente envía la petición al nodo servidor, con el estilo de intercambio y las medidas de fiabilidad aplicados.	
(bloquea en espera de la respuesta)	El *dispatcher* del servidor recibe la petición. El identificador de operación incluido en el mensaje indica qué procedimiento o método se ha de ejecutar.
	El *stub* del servidor deserializa los argumentos contenidos en el mensaje, reconstruyendo a partir de ellos los valores nativos que necesita la operación.
	El *stub* del servidor invoca localmente la implementación real de la operación.
	Al termianr la ejecución, el *stub* del servidor serializa el resultado —o el error, si la operación falló— en un mensaje de respuesta.
El *stub* recibe la respuesta, deserializa el resultado y lo devuelve como valor de retorno de la llamada original.	

CUADRO 23.1: Pasos de una invocación remota

El identificador de la operación (paso 4) le indica al servidor qué operación quiere ejecutar. No se debe confundir con el identificador de mensaje (§ 23.1), que se utiliza para distinguir ocurrencias distintas, incluso aunque correspondan a la misma operación.

El mecanismo de invocación remota esencialmente ofrece transparencia: la invocación al método remoto es muy similar sintácticamente a una llamada local equivalente. Sin embargo, hay varias diferencias que no se pueden (y no se deben) ocultar: latencia, errores en el servidor, fallo parcial o las garantías de entrega aplicadas (§ 23.4).

23.5.1. pico-rpc

Pico-rpc es un middleware RPC mínimo escrito en Python y con fines exclusivamente docentes. Implementa las piezas elementales del mecanismo de invocación remota: lenguaje de descripción de interfaces, compilador, generación de los *stubs* de cliente y servidor, y *dispatcher*; pero no implementa la mayoría del control de errores o mecanismos de seguridad. Con pico-rpc resulta sencillo hacer el recorrido completo de la descripción de RPC de Coulouris *et al.* [63]. Consta de los siguientes ficheros:

- `picorpc.py` es el *runtime* del middleware. Contiene las primitivas para describir interfaces y el *dispatcher* del servidor.
- `rpcgen.py` es el compilador de interfaces. Genera los *stubs* a partir de la definición de la interfaz.
- `math.idl.py` es la definición de la interfaz del ejemplo.
- `server.py` corresponde con el servidor de la aplicación, que incluye implementación de los procedimientos remotos que aporta el programador de la aplicación. Por último `client.py` es el cliente que realiza la invocación.

Interfaz

Normalmente, la definición de la interfaz se escribe con un lenguaje específico y puramente declarativo, llamado IDL, que es independiente del lenguaje de programación en el que se implementa cliente y servidor. El compilador genera los *stubs* en el lenguaje de destino a partir de esa definición. Por simplicidad, en pico-rpc, la definición de la interfaz se escribe también en Python mediante una clase que hereda de `Interface`, y un método sin cuerpo por cada operación. Las anotaciones de tipo de los parámetros y del valor de retorno determinan cómo se serializa cada valor (Listado 23.1).

```python
from picorpc import Interface, uint8, uint64

class Math(Interface):
    def factorial(n: uint8) -> uint64: ...
    def power(base: uint8, exp: uint8) -> uint64: ...
```

Listado 23.1: Interfaz del servicio `Math`
/pico-rpc/math.idl.py

Los tipos `uint8` y `uint64` corresponden directamente con los caracteres de formato de `struct` para enteros sin signo de 1 y 8 bytes (`'B'` y `'Q'`). El identificador de cada operación es su posición en la interfaz: 0 para `factorial()` y 1 para `power()`.

El «compilador» lee esta definición y genera los *stubs* de cliente y servidor:

```
$ ./rpcgen.py math.idl.py
generated math_client_stub.py and math_server_stub.py from math.idl.py
```

Cliente

El *stub* de cliente es una clase (`MathStub`) con un método por cada operación de la interfaz (Listado 23.2). Cada método abre una nueva conexión TCP, construye el mensaje de petición con el identificador de la operación (1 byte) seguido de los argumentos serializados, y espera la respuesta, que contiene el resultado serializado como un entero de 8 bytes. Esto encaja con el estilo RR en el que cada conexión transporta una única petición y su respuesta. No hace falta un identificador de mensaje para asociarlas, porque la conexión no se reutiliza y TCP resuelve las retransmisiones de segmentos sin que la aplicación reciba duplicados.

```
class MathStub:
    FACTORIAL = 0
    POWER = 1

    def __init__(self, host, port):
        self.server_endpoint = host, port

    def factorial(self, n):
        sock = socket.socket()
        sock.connect(self.server_endpoint)

        request = struct.pack('BB', self.FACTORIAL, n)
        sock.sendall(request)

        reply = sock.recv(MAX_MESSAGE_SIZE)
        result = struct.unpack('Q', reply)[0]

        sock.close()
        return result
```

LISTADO 23.2: *Stub* de cliente generado (fragmento)
`math_client_stub.py`

El programa cliente solo necesita instanciar el *stub* indicando el *endpoint* del servidor e invocar sus métodos del mismo mode que lo haría con un objeto convencional (Listado 23.3).

```
import sys

from math_client_stub import MathStub

if len(sys.argv) != 3:
    print("usage: ./client <server> <arg>")
```

```
    exit(1)

server = sys.argv[1]
arg = int(sys.argv[2])

stub = MathStub(server, 2000)
print("factorial({}) = '{}'".format(arg, stub.factorial(arg)))
print("power({}, {}) = '{}'".format(arg, arg-1, stub.power(arg, arg-1)))
```

LISTADO 23.3: Cliente del servicio `Math`
`pico-rpc/client.py`

Servidor

En el lado del servidor, el compilador genera una función *stub* por cada operación y una tabla (`STUBS`) que asocia cada identificador de operación con su función *stub* (Listado 23.4). Cada una de esas funciones deserializa los parámetros, invoca la implementación real de la operación y serializa el resultado.

```
def factorial_stub(args, implementation):
    args = struct.unpack('B', args)
    result = implementation.factorial(*args)
    return struct.pack('Q', result)

def power_stub(args, implementation):
    args = struct.unpack('BB', args)
    result = implementation.power(*args)
    return struct.pack('Q', result)

FACTORIAL = 0
POWER = 1

STUBS = {
    FACTORIAL: factorial_stub,
    POWER: power_stub,
}
```

LISTADO 23.4: *Stub* de servidor generado
`math_server_stub.py`

La multiplexación la realiza el *dispatcher* (clase `Dispatcher` de `picorpc.py`). Acepta conexiones de forma iterativa (como en § 15.2), lee el mensaje de petición, utiliza su primer byte para localizar la función *stub* correspondiente en la tabla `STUBS` y le entrega el resto del mensaje, es decir, los argumentos serializados. Por último, envía al cliente el resultado que devuelve el *stub*.

En el servidor el programador define una clase que implementa las operaciones de la interfaz (`MathI`), y registra una instancia en el *dispatcher* junto con el módulo del *stub* de servidor (Listado 23.5).

```python
from picorpc import Dispatcher
import math_server_stub

class MathI:
    def factorial(self, n):
        if n == 0:
            return 1

        return n * self.factorial(n - 1)

    def power(self, base, exp):
        return base ** exp

dispatcher = Dispatcher(port=2000)
dispatcher.register(math_server_stub, MathI())
dispatcher.run()
```

Listado 23.5: Servidor del servicio `Math`
`/pico-rpc/server.py`

Es importante aclarar que a pesar de que pico-rpc utiliza clases tanto para la definición de la interfaz y para la implementación de las operaciones, lo hace solo por conveniencia de implementación. Sus métodos no tienen relación entre sí y funcionan como operaciones independientes.

Ejecución del servidor y del cliente en el mismo computador:

Cliente

```
$ ./client.py 127.0.0.1 5
factorial(5) = '120'
power(5, 4) = '625'
```

Servidor

```
$ ./server.py
Server ready: ('0.0.0.0', 2000)
```

Figura 23.2: Servidor y cliente pico-rpc

Para mantener el código al mínimo, pico-rpc prescinde deliberadamente de aspectos que ningún middleware real puede ignorar. Solo admite enteros sin signo, supone que cada mensaje llega completo en una única llamada a `recv()` —ignora la E/S parcial— el servidor es iterativo y no aplica ninguna medida de tolerancia a fallos más allá de las que proporciona TCP. Tampoco transporta errores de vuelta al cliente si la ejecución de la operación o la serialización falla, *p. ej.* el resultado de `factorial(21)` no cabe en un `uint64`.

23.5.2. ONC RPC

ONC RPC—también llamado Sun RPC— fue uno de los middlewares pioneros de la invocación remota. Lo desarrolló Sun Microsystems en los años

80 y sobre él está construido el popular sistema de ficheros en red NFS que aún hoy día se sigue utilizando.

Serializa los datos con un formato llamado XDR [64], y las interfaces de los procedimientos remotos se describen en ficheros `.x`, un lenguaje que extiende XDR para declarar programas y procedimientos. La implementación original de Sun, de la que derivan las actuales, solo permite escribir clientes y servidores en C, aunque existen implementaciones independientes para otros lenguajes.

El ejemplo /`onc-rpc-factorial` es un servicio equivalente al de pico-rpc, aunque únicamente con la operación `factorial()`. La interfaz (Listado 23.6) declara un «programa», identificado por un número (`600000002`), con una versión (`1`) que contiene el procedimiento `FACTORIAL`, también numerado (`1`). El número de procedimiento cumple la misma función que el identificador de operación de pico-rpc, pero también en otros middlewares reales.

```
program FACTORIAL_PROG {
   version FACTORIAL_VERS {
    unsigned long FACTORIAL(int n) = 1;
   } = 1;
} = 600000002;
```

Listado 23.6: Interfaz del servicio `FACTORIAL_PROG`
/`onc-rpc-factorial/factorial.x`

A partir de la interfaz, `rpcgen` (`rpcsvc-proto`) genera tres ficheros: `factorial.h`, con las constantes y los prototipos; `factorial_clnt.c`, el *stub* de cliente, y `factorial_svc.c`, el *stub* de servidor, que incluye el *dispatcher* y la función `main()` del servidor. El programador aporta lo demás:

- `factorial_server.c` es la implementación del procedimiento en la función `factorial_1_svc()`, cuyo nombre se forma con el del procedimiento y el número de versión. Devuelve un puntero a una variable estática porque el *stub* serializa el resultado después de que la función haya retornado (Listado 23.7).
- `factorial_client.c` obtiene un manejador de cliente con `clnt_create()`, indicando el nodo, el programa, la versión y el transporte (UDP), e invoca la operación con `factorial_1()`, que es la función del *stub* de cliente (Listado 23.8).

```
u_long *factorial_1_svc(int *n, struct svc_req *rqstp) {
    static u_long result;
    result = 1;

    printf("Received: n=%d \n", *n);
```

```
    for (int i=*n; i>0; i--)
        result *= i;

    return &result;
}
```

LISTADO 23.7: Implementación del procedimiento `FACTORIAL`
/onc-rpc-factorial/factorial_server.c

```
void factorial_prog_1(char *host, int n) {
    CLIENT *clnt;
    u_long *result_1;

#ifndef DEBUG
    clnt = clnt_create (host, FACTORIAL_PROG, FACTORIAL_VERS, "udp");
    if (clnt == NULL) {
        clnt_pcreateerror (host);
        exit (1);
    }
#endif

    result_1 = factorial_1(&n, clnt);
    if (result_1 == NULL)
        clnt_perror(clnt, "call failed");
    else
        printf("factorial(%d): %lu", n, *result_1);

#ifndef DEBUG
    clnt_destroy (clnt);
#endif
}
```

LISTADO 23.8: Cliente del servicio `FACTORIAL_PROG`
/onc-rpc-factorial/factorial_client.c

Al arrancar, el servidor crea un socket UDP y otro TCP, sin puerto fijo, y los registra en `rpcbind` (versión actual del antiguo «portmapper»), un servicio que asocia cada combinación de programa, versión y transporte con el puerto en el que está disponible.

El servidor ofrece los dos transportes para que el cliente pueda elegir cuál usar. Para eso se utiliza el último argumento de `clnt_create()` (Listado 23.8). Lógicamente cada transporte implica una semántica de invocación distinta (§ 23.4). La función `clnt_create()` consulta a `rpcbind` en el nodo remoto para averiguar el puerto que corresponde al transporte elegido.

- Con UDP el *runtime* del cliente retransmite la petición si la respuesta no llega a tiempo, y cada mensaje lleva un identificador de transacción (`xid`) para asociar la respuesta con su petición. Sin embargo, el servidor no filtra duplicados salvo que se active expresamente su caché de peticiones, así que la semántica es *at-least-once*. Con la caché

activa, el servidor reenvía la respuesta guardada en lugar de ejecutar de nuevo el procedimiento, lo que se aproxima a *at-most-once*, aunque la caché tiene un tamaño limitado y se pierde si el servidor se reinicia.

- Con TCP el *runtime* del cliente no retransmite. La petición se envía una sola vez y es TCP quien garantiza que llega sin pérdidas ni duplicados. Si la conexión se interrumpe o expira la espera, la llamada devuelve un error sin reintentarlo y el cliente no puede saber si el procedimiento llegó a ejecutarse. El procedimiento se ejecuta, por tanto, como mucho una vez: la semántica es *at-most-once*.

El cliente del ejemplo utiliza UDP. Para este servicio es suficiente, porque calcular el factorial es una operación idempotente.

El ejemplo /`onc-rpc-hello` es aún más básico. Solo define un procedimiento que recibe una cadena y la imprime en el servidor. Está tomado, con algunas simplificaciones, de la documentación oficial de ONC RPC [65]. Incluye el mismo programa en dos versiones, una local con una llamada a función convencional (directorio `local`) y otra en la que esa función pasa a ser un procedimiento remoto (directorio `remote`), lo que permite comparar qué cambia al convertir una llamada local en remota.

23.5.3. gRPC

gRPC es un middleware RPC de código abierto que Google publicó en 2015 a partir de su experiencia con su sistema interno de invocación remota. Actualmente es uno de los más utilizados para comunicar servicios dentro de la infraestructura interna o un centro de datos, sobre todo para arquitecturas de microservicios.

Las interfaces se describen en ficheros con extensión `.proto` en el lenguaje Protocol Buffers, que es a la vez el IDL y el formato de serialización binario. A diferencia de ONC RPC, el compilador genera *stubs* para muchos lenguajes (C++, Java, Go, Python, etc.), de modo que cliente y servidor se pueden escribir en lenguajes distintos. Como transporte utiliza HTTP/2 sobre TCP, y opcionalmente acepta cifrado con TLS.

El ejemplo /`grpc-factorial` es equivalente al ejemplo de ONC RPC, salvo que en este caso se puede escribir en Python. La interfaz (Listado 23.9) declara un «servicio» `Math` con la operación `factorial()`. Una operación unaria de gRPC recibe exactamente un mensaje y devuelve exactamente otro, por lo que el parámetro y el resultado se definen como mensajes (`FactorialRequest` y `FactorialReply`), aunque cada uno contenga un único campo. El número que acompaña a cada campo (`= 1`) es el que lo identifica en el mensaje serializado. Eso permite añadir campos nuevos a un mensaje

sin romper la compatibilidad con clientes o servidores que usan la versión anterior de la interfaz, algo que Google consideró una característica esencial en el diseño de Protocol Buffers.

```
syntax = "proto3";

package math;

service Math {
  rpc factorial (FactorialRequest) returns (FactorialReply) {}
}

message FactorialRequest {
  int32 value = 1;
}

message FactorialReply {
  int64 result = 1;
}
```

LISTADO 23.9: Interfaz del servicio `Math` en gRPC
/`grpc-factorial/math.proto`

El compilador `protoc`, que para Python se ejecuta como el módulo `grpc\_tools` (`python3-grpc-tools`), genera dos ficheros a partir de la interfaz: `math_pb2.py`, con las clases de los mensajes y su serialización, y `math_pb2_grpc.py`, con los *stubs* de cliente (`MathStub`) y servidor (`MathServicer`). El programador aporta lo demás:

- `server.py` implementa la operación en una clase que hereda de `MathServicer` (Listado 23.10). El método `factorial()` recibe el mensaje de petición ya deserializado y un contexto, que da acceso a los metadatos de la llamada y permite, entre otras cosas, devolver un código de error. La función `add_MathServicer_to_server()` registra esa implementación en el servidor, que la asocia con las operaciones de la interfaz. Es el equivalente a la tabla `STUBS` de pico-rpc.
- `client.py` crea un canal hacia el *endpoint* del servidor, instancia el *stub* sobre ese canal e invoca la operación con un objeto `FactorialRequest` como argumento (Listado 23.11).

```
from concurrent import futures

import grpc
import math_pb2
import math_pb2_grpc

class Math(math_pb2_grpc.MathServicer):
    def factorial(self, request, context):
        result = 1
```

```
        n = request.value

        while n:
            result *= n
            n -= 1

        return math_pb2.FactorialReply(result=result)

server = grpc.server(futures.ThreadPoolExecutor(max_workers=10))
math_pb2_grpc.add_MathServicer_to_server(Math(), server)
server.add_insecure_port('[::]:2000')
server.start()

try:
    server.wait_for_termination()

except KeyboardInterrupt:
    server.stop(0)
```

LISTADO 23.10: Servidor del servicio `Math` en gRPC
/grpc-factorial/server.py

```
import sys

import grpc
import math_pb2
import math_pb2_grpc

if len(sys.argv) != 3:
    print("usage: ./client <server> <value>")
    sys.exit(1)

server = sys.argv[1]
value = int(sys.argv[2])

channel = grpc.insecure_channel('{}:2000'.format(server))
stub = math_pb2_grpc.MathStub(channel)
response = stub.factorial(math_pb2.FactorialRequest(value=value))
print("result = '{}'".format(response.result))
```

LISTADO 23.11: Cliente del servicio `Math` en gRPC
/grpc-factorial/client.py

El servidor escucha en un puerto TCP indicado (2000). No existe ningún equivalente a `rpcbind`, el cliente debe conocer de antemano el `d:el` servidor. Tanto `insecure_channel()` en el cliente como `add_insecure_port()` en el servidor indican que la comunicación no está cifrada. En un despliegue real se aconseja usar sus equivalentes con TLS. Además, a diferencia del de pico-rpc, el servidor es concurrente; atiende las peticiones con un *pool* de hasta 10 hilos (`ThreadPoolExecutor`).

Cada invocación se envía como una petición `POST` de HTTP/2 dirigida a una ruta formada por el nombre del servicio y el de la operación (`/math.Math/factorial`). El cuerpo contiene el mensaje serializado precedido de 5 bytes:

uno que indica si va comprimido y cuatro con su longitud. El resultado de la ejecución —éxito o un código de error— se devuelve en las cabeceras finales de la respuesta. HTTP/2 permite multiplexar varias invocaciones simultáneas sobre la misma conexión TCP, así que el cliente no necesita abrir una conexión por cada llamada, como hace pico-rpc.

Respecto a la semántica de invocación, el *runtime* del cliente no reintenta por defecto una petición que haya podido llegar al servidor, y TCP evita pérdidas y duplicados. Si la llamada falla o expira su plazo, el cliente recibe un error y no puede saber si la operación llegó a ejecutarse, de modo que, igual que ONC RPC sobre TCP, la semántica es *at-most-once*. Los reintentos automáticos se pueden activar, pero en ese caso es responsabilidad del programador determinar si la operación es idempotente.

Por supuesto, gRPC transporta los errores hasta el cliente. Si la implementación de `factorial()` lanza una excepción —*p. ej.* porque el resultado de `factorial(21)` no cabe en el campo `int64` de `FactorialReply`— el cliente recibe un error con el código de estado `UNKNOWN` y el texto de la excepción, en lugar de quedarse esperando o recibir un mensaje corrupto.

Por último, además de las invocaciones «unarias» como la del ejemplo (una petición y una respuesta), gRPC admite otras tres modalidades en las que uno o ambos extremos envían un flujo de mensajes:

- *server streaming*, en el que el servidor devuelve una secuencia de respuestas a una única petición;
- *client streaming*, en el que el cliente envía una secuencia de mensajes y recibe una única respuesta; y
- *bidireccional*, en la que ambos envían secuencias de forma independiente.

Estos modos se declaran en la interfaz anteponiendo la palabra clave `stream` al tipo del parámetro, al del resultado o a ambos.

El ejemplo /`grpc-streaming` amplía el servicio `Math` con una operación para cada una de esas modalidades (Listado 23.12): `factorials()` obtiene un valor límite y el servidor devuelve la secuencia de factoriales hasta ese límite; `sum()` recibe una secuencia de enteros y devuelve su suma; y `factorial_each()` devuelve el factorial de cada valor que recibe.

```
syntax = "proto3";

package math;

service Math {
```

```
  // the server returns 1!, 2!, ..., n!
  rpc factorials (Limit) returns (stream Number) {}

  // the client sends numbers and the server returns their sum
  rpc sum (stream Number) returns (Number) {}

  // the server returns the factorial of each number as it arrives
  rpc factorial_each (stream Number) returns (stream Number) {}
}

message Limit {
  int32 value = 1;
}

message Number {
  int64 value = 1;
}
```

Listado 23.12: Interfaz del servicio `Math` con flujos
`/grpc-streaming/math.proto`

En Python esos flujos se implementan con iteradores, de modo que el *stub* traduce cada modalidad al mecanismo de generadores (Listado 23.13). Cuando el resultado es un flujo, la implementación es un generador que entrega un mensaje cada vez con `yield`, y el *runtime* lo envía en cuanto se produce. Cuando el parámetro es un flujo, la implementación recibe iterador (`request_iterator`) del que extrae los mensajes conforme llegan; ese iterador se agota cuando el cliente cierra su mitad del flujo.

```
import math
import time
from concurrent import futures

import grpc
import math_pb2
import math_pb2_grpc

class Math(math_pb2_grpc.MathServicer):
    def factorials(self, request, context):
        for n in range(1, request.value + 1):
            time.sleep(0.5)
            yield math_pb2.Number(value=math.factorial(n))

    def sum(self, request_iterator, context):
        total = 0
        for number in request_iterator:
            total += number.value

        return math_pb2.Number(value=total)

    def factorial_each(self, request_iterator, context):
        for number in request_iterator:
            yield math_pb2.Number(value=math.factorial(number.value))

server = grpc.server(futures.ThreadPoolExecutor(max_workers=10))
```

```
math_pb2_grpc.add_MathServicer_to_server(Math(), server)
server.add_insecure_port('[::]:2000')
server.start()

try:
    server.wait_for_termination()

except KeyboardInterrupt:
    server.stop(0)
```

Listado 23.13: Servidor con las tres modalidades de flujo
/grpc-streaming/server.py

En el cliente, la correspondencia es la opuesta (Listado 23.14): la invocación de una operación que devuelve un flujo devuelve en realidad un iterador que el cliente recorre, y una operación cuyo parámetro es un flujo admite un iterable como argumento, en este caso el generador `numbers()`.

```
def numbers(values):
    for value in values:
        yield math_pb2.Number(value=value)

print("server streaming:")
for reply in stub.factorials(math_pb2.Limit(value=limit)):
    print("  ", reply.value)

print("client streaming:")
reply = stub.sum(numbers(range(1, limit + 1)))
print("  ", reply.value)

print("bidirectional streaming:")
for reply in stub.factorial_each(numbers(range(1, limit + 1))):
    print("  ", reply.value)
```

Listado 23.14: Cliente con las tres modalidades de flujo
/grpc-streaming/client.py

La diferencia respecto a encadenar varias invocaciones unarias no es solo sintáctica. Todos los mensajes de un mismo flujo se envían sobre el mismo *stream* HTTP/2, que es la forma en la que gRPC delimita una invocación. Hay una sola petición, un solo plazo *(deadline)* para toda la operación y un único código de estado final, que llega en las cabeceras al cerrarse el flujo. Pero cada mensaje se entrega al receptor en cuanto el emisor lo produce, sin esperar a los demás. El retardo artificial de medio segundo de `factorials()` evidencia ese comportamiento. El cliente imprime cada resultado delfactorial conforme lo recibe, no todos de golpe al terminar.

En la modalidad bidireccional los dos flujos son independientes. El servidor no está obligado a consumir todos los mensajes antes de empezar a responder, ni a emitir uno por cada uno que recibe. La función `factorial_each()` implementa la alternancia más simple —un resultado por cada argumento—

, pero igualmente podría agrupar varios, descartar algunos o emitir mensajes sin haber recibido nada.

23.5.4. JSON-RPC

JSON-RPC es un protocolo RPC cuyos mensajes son objetos JSON. A pesar de su sencillez, o quizá precisamente por eso, tiene bastante aceptación. Sobre él se construyen, por ejemplo, MCP (Model Context Protocol), con el que las aplicaciones basadas en LLM descubren e interaccionan con otros programas, y LSP, con el que los editores de código solicitan a un servidor especializado en un lenguaje de programación cosas como completar identificadores o marcar errores del código. También lo emplean las API de los nodos de la red Ethereum. En MCP y LSP ambos extremos emiten peticiones.

La especificación de su versión 2.0 [66] es un documento breve, que se limita a definir el formato de los mensajes y la correspondencia entre cada respuesta y su petición. No existe un IDL, un compilador ni *stubs*, ni tampoco un *runtime* de referencia. La especificación tampoco establece el transporte ni el modo de delimitar los mensajes sobre él, de manera que un mensaje puede transportarse en el cuerpo de una petición HTTP, en una línea de texto sobre TCP o precedido de una cabecera que indique su longitud, como hace LSP. Adopta el mismo planteamiento que XML-RPC, un protocolo anterior, con JSON en lugar de XML.

La invocación `subtract(42, 23)` y una invocación a un método inexistente dan lugar al siguiente intercambio, expresado en la notación de la propia especificación, en la que `-->` señala el mensaje que envía el cliente y `<--` el que devuelve el servidor:

```
--> {"jsonrpc": "2.0", "method": "subtract", "params": [42, 23], "id": 1}
<-- {"jsonrpc": "2.0", "result": 19, "id": 1}
--> {"jsonrpc": "2.0", "method": "foobar", "params": [42], "id": 2}
<-- {"jsonrpc": "2.0", "error": {"code": -32601, "message": "Method not found"}, "id": 2}
```

Una petición es un objeto con cuatro miembros. `jsonrpc` indica la versión (siempre la cadena `2.0`), `method` es el nombre de la operación, `params` contiene los argumentos, ya sea por posición (un *array*) o por nombre (un objeto), e `id` identifica la petición. La respuesta incluye el mismo `id` y, además, un miembro `result` con el resultado o un miembro `error` con un código numérico y un mensaje. Ese `id` corresponde al identificador de mensaje de § 23.1, y permite al cliente mantener varias peticiones en curso sobre la misma conexión y asociar cada respuesta a la petición correspondiente. El identificador de operación, en cambio, es el nombre del método, es decir, una cadena, y no un número como en pico-rpc o en ONC RPC.

Una petición que carece de `id` es una *notificación*. El servidor la ejecuta sin enviar respuesta alguna, ni siquiera en caso de error, por lo que corresponde al estilo R y el cliente no puede saber si la operación llegó a ejecutarse. Asimismo, es posible agrupar varias peticiones en un *array*, que constituye un *lote (batch)*. El servidor responde con otro *array* que contiene las respuestas a todas las peticiones salvo a las notificaciones, en cualquier orden, puesto que cada una incluye su `id`. Un lote reduce el número de tiempos de ida y vuelta cuando las peticiones son independientes entre sí.

El ejemplo /`jsonrpc-factorial` utiliza la biblioteca `jsonrpclib` (`python3-jsonrpclib-pelix`). Ofrece la operación `factorial()` de los ejemplos anteriores y otra, `log()`, que imprime un texto en la consola del servidor y sirve para ilustrar las notificaciones. Cada mensaje se envía en el cuerpo de una petición `POST` de HTTP, siempre a la misma URI, de modo que, igual que en gRPC, HTTP actúa únicamente como transporte.

El servidor registra cada función con `register_function()`, que la asocia a su nombre, y para despachar una petición busca ese nombre en un diccionario, del mismo modo que la tabla `STUBS` de pico-rpc. No se requiere un *stub* de servidor, ya que el módulo `json` entrega los argumentos como valores nativos de Python. El servidor, en cambio, debe validar todo lo que recibe, porque nada garantiza que la petición tenga la forma esperada. La especificación reserva un código de error para cada caso, *p. ej.* `-32601` si el método no existe o `-32602` si los argumentos no son válidos.

Tampoco el cliente dispone de *stubs*. La clase `ServerProxy` construye cada petición a partir del nombre del método invocado, de modo que `server.factorial(5)` se convierte en una invocación remota sin que la operación se haya declarado previamente en ninguna interfaz. La contrapartida de esa flexibilidad es que los errores que se detectarían al generar los *stubs* a partir de una interfaz solo se manifiestan en tiempo de ejecución, y es el servidor quien los detecta.

Puesto que los mensajes son texto, cualquier cliente HTTP, como `curl`, puede efectuar invocaciones:

```
$ curl localhost:2000 \
  -d '{"jsonrpc": "2.0", "method": "factorial", "params": [5], "id": 1}'
{"result": 120, "id": 1, "jsonrpc": "2.0"}
$ curl localhost:2000 \
  -d '{"jsonrpc": "2.0", "method": "factorial", "params": {"n": 20}, "id": 2}'
{"result": 2432902008176640000, "id": 2, "jsonrpc": "2.0"}
$ curl localhost:2000 \
  -d '{"jsonrpc": "2.0", "method": "log", "params": ["hello"]}'
$ curl localhost:2000 -d '[
  {"jsonrpc": "2.0", "method": "factorial", "params": [3], "id": 3},
  {"jsonrpc": "2.0", "method": "power", "params": [2, 8], "id": 4}]'
```

```
[{"result": 6, "id": 3, "jsonrpc": "2.0"},
 {"id": 4, "jsonrpc": "2.0",
  "error": {"code": -32601, "message": "Method power not supported."}}]
```

La segunda invocación proporciona el argumento por nombre. La tercera es una notificación, de modo que el servidor contesta a la petición HTTP, como exige ese protocolo, pero sin ningún mensaje JSON-RPC en el cuerpo. La cuarta es un lote de dos peticiones, la segunda de ellas dirigida a un método inexistente.

JSON no distingue entre enteros y reales, y muchas implementaciones representan los números como reales de doble precisión, así que un entero mayor que 2^{53} puede llegar redondeado [67]. En gRPC, en cambio, el tipo `int64` de la interfaz produce un error si el valor no cabe, en lugar de entregar uno incorrecto.

23.6. RMI

RMI traslada el mecanismo de invocación remota al modelo de objetos. La arquitectura es la misma que en RPC—interfaz, compilador, *stubs* y *runtime*—, pero el destino de una invocación no es un procedimiento del servidor, sino un método de un objeto concreto. Esa diferencia, aparentemente pequeña, tiene varias consecuencias.

En RPC, el servidor ofrece un conjunto plano de procedimientos, y toda invocación de una operación llega a la misma implementación. Si el servicio gestiona varias entidades independientes —varios ficheros, varias cuentas o varios contadores— el procedimiento no tiene forma de distinguirlas por sí mismo; el cliente tiene que incluir en cada invocación un argumento adicional que identifique la entidad, y el servidor tiene que mantener una tabla que asocie ese identificador con su estado. Eso es lo que hace NFS, por ejemplo, con los manejadores de fichero.

En RMI, en cambio, cada objeto remoto encapsula su propio estado, y el cliente invoca los métodos directamente sobre el objeto que le interesa. Un mismo servidor puede alojar muchos objetos que implementan la misma interfaz: dos contadores remotos llevan cuentas independientes aunque compartan servidor, tipo y código, igual que dos instancias de una misma clase en un programa convencional.

Para lograrlo, cada objeto remoto tiene una **referencia a objeto remoto**, que lo identifica de forma única en todo el sistema. De un modo u otro, esta referencia remota incluye el *endpoint* del servidor que lo aloja y una *identidad* que identifica el objeto. En el cliente, la referencia se materializa en un *proxy*, que es parte del *stub* de cliente y está ligado a un objeto concreto de

ese servidor. La invocación, por tanto, requiere dos niveles de despachado[2] en el servidor. Primero un componente del *runtime*, habitualmente llamado *adaptador de objetos*, localiza el objeto a partir de su identidad. Después, el *stub* de servidor de ese objeto localiza el método a partir del identificador de operación. El objeto que implementa la interfaz en el servidor se suele denominar *sirviente (servant)*.

La ventaja más importante de este modelo es que las referencias a objetos remotos son «ciudadanos de primera clase» *(first-class citizens)*, es decir, valores que se pueden pasar como argumento o devolver como resultado de una invocación, igual que una referencia a un objeto convencional. Eso tiene varias consecuencias:

- El conjunto de objetos accesibles no está fijado de antemano. Un servidor puede crear objetos nuevos durante la ejecución y devolver sus referencias al cliente, *p. ej.* un objeto «factoría» cuyo método crea una sesión y devuelve la referencia al nuevo objeto.
- Los parámetros admiten dos modos de paso. Los objetos remotos se pasan por referencia: el receptor obtiene un proxy y sus invocaciones llegan al objeto original. El resto de valores (números, cadenas, estructuras) se pasan por valor: se serializan y el receptor obtiene una copia. En RPC solo existe el paso por valor.
- El cliente puede pasar al servidor la referencia de un objeto propio para que el servidor lo invoque más tarde, *p. ej.* para notificarle un evento. Esto es lo que llamamos *callback*, y es una de las situaciones en las que los roles de cliente y servidor de la invocación se invierten para una interacción concreta.

Por lo demás, las interfaces de RMI heredan las características del modelo de objetos del lenguaje: pueden extender otras interfaces (herencia de interfaz), y un proxy de una interfaz derivada se puede usar allí donde se espera una base *(upcasting)*. Los errores del servidor se propagan como excepciones, que el *stub* de cliente vuelve a lanzar en el cliente.

Todo esto por supuesto tiene un coste. La vida de un objeto remoto debería terminar cuando ningún cliente lo necesita, pero el servidor no puede saber cuándo han dejado de existir los *proxies* que lo referencian en otros nodos. Algunos middlewares incorporan para eso un recolector de basura distribuido; otros dejan esa responsabilidad al programador, que tiene que destruir explícitamente los objetos que ya no necesita. Además, la facilidad para crear y repartir referencias invita a diseños con muchas invocaciones

[2] también llamado «indirección»

pequeñas, en las que la latencia de la red —que el modelo de objetos no oculta— acaba suponiendo una sobrecarga no trivial.

23.6.1. pico-rmi

Pico-rmi es la versión orientada a objetos de pico-rpc (§ 23.5.1). Mantiene la misma estructura —*runtime* (`picormi.py`), compilador (`rmigen.py`), definición de la interfaz en Python y serialización con `struct`— y solo cambia lo necesario para que un servidor pueda alojar varios objetos independientes.

La interfaz `Counter`, del ejemplo /`pico-rmi`, declara dos operaciones sin parámetros: `increment()` y `get()`. La mayor diferencia respecto a pico-rpc no está en la interfaz, sino en que ahora puede haber varios objetos que la implementan.

Cliente

El *stub* del cliente para un objeto remoto concreto se llama *proxy*, y para instanciarlo se requiere, además del *endpoint* del servidor, la identidad del objeto. Esta combinación es la «referencia de objeto remoto» o simplemente «referencia remota».

```
from counter_client_stub import CounterProxy

if len(sys.argv) != 3:
    print("usage: ./client <server> <identity>")
    exit(1)

server = sys.argv[1]
identity = sys.argv[2]

counter = CounterProxy(identity, server, 2001)
print("get() = '{}'".format(counter.get()))
print("increment() = '{}'".format(counter.increment()))
print("increment() = '{}'".format(counter.increment()))
print("get() = '{}'".format(counter.get()))
```

Listado 23.15: Cliente del ejemplo `Counter` de pico-rmi
/`pico-rmi/client.py`

El mensaje de petición incluye en este caso la identidad del objeto (16 bytes) seguida del identificador de operación (1 byte), y los argumentos si los hubiera.

```
request = struct.pack('16sB', self.identity, self.INCREMENT)
```

Listado 23.16: Mensaje de petición pico-rmi generado en el *stub* del cliente

Servidor

En pico-rpc el *stub* de servidor es un módulo con funciones sueltas que reciben la implementación como argumento. En pico-rmi el compilador genera una clase (`Counter`) que hereda de `Servant`, con un método *stub* por cada operación y la misma tabla `STUBS`. El sirviente (`CounterI`) hereda de esa clase, así que cada *stub* invoca la operación sobre la propia instancia (`self`) y, por tanto, sobre su estado.

El *dispatcher* de pico-rpc se sustituye por el «adaptador de objetos» según la terminología habitual de RMI, que mantiene un diccionario de sirvientes indexado por la identidad del objeto (Listado 23.17). Al recibir una petición, obtiene la identidad y localiza el sirviente, que a su vez localiza el *stub* a partir del identificador de operación. Estos son los dos niveles de despachado descritos al principio de esta sección.

```
class Servant:
    STUBS: dict[int, Callable] = {}

    def dispatch(self, function_id, args):
        return self.STUBS[function_id](self, args)

class ObjectAdapter:
    def __init__(self, port):
        self.port = port
        self.servants = {}

    def add(self, identity, servant):
        self.servants[identity] = servant

    def dispatch(self, sock):
        request = sock.recv(MAX_MESSAGE_SIZE)
        identity = decode_identity(request[:IDENTITY_SIZE])
        function_id, args = request[IDENTITY_SIZE], request[IDENTITY_SIZE + 1:]

        servant = self.servants[identity]
        result = servant.dispatch(function_id, args)
        sock.sendall(result)
```

LISTADO 23.17: Sirviente base y adaptador de objetos de *runtime* de pico-rmi
/pico-rmi/picormi.py

El servidor crea dos sirvientes y los registra en el adaptador con las identidades `counter1` y `counter2` (Listado 23.18).

```
from picormi import ObjectAdapter
from counter_server_stub import Counter

class CounterI(Counter):
    def __init__(self):
        self.value = 0
```

```
    def increment(self):
        self.value += 1
        return self.value

    def get(self):
        return self.value

adapter = ObjectAdapter(port=2001)
adapter.add('counter1', CounterI())
adapter.add('counter2', CounterI())
adapter.run()
```

LISTADO 23.18: Servidor del servicio `Counter`
`pico-rmi/server.py`

Cada ejecución del cliente invoca `get()`, dos veces `increment()` y de nuevo `get()` sobre el objeto cuya identidad recibe como argumento. Los dos contadores llevan cuentas independientes, aunque estén en el mismo servidor y compartan la misma interfaz. En pico-rpc eso habría requerido añadir un parámetro que identificara sobre qué contador debe aplicar la operación.

Cliente

```
$ ./client.py 127.0.0.1 counter1
get() = '0'
increment() = '1'
increment() = '2'
get() = '2'
$ ./client.py 127.0.0.1 counter1
get() = '2'
increment() = '3'
increment() = '4'
get() = '4'
$ ./client.py 127.0.0.1 counter2
get() = '0'
increment() = '1'
increment() = '2'
get() = '2'
```

Servidor

```
$ ./server.py
Object adapter ready: ('0.0.0.0', 2001)
```

FIGURA 23.3: Cliente y servidor pico-rmi

Pico-rmi aplica todas las simplificaciones de pico-rpc, y añade otras propias del modelo de objetos. Los sirvientes se registran al arrancar el servidor y no se pueden crear ni destruir después. Tampoco es posible pasar una referencia de objeto remoto como argumento o como valor de retorno, porque solo admite enteros sin signo. Por último, si la identidad de la petición no corresponde a ningún sirviente, el adaptador no informa al cliente del error: la excepción termina el servidor.

23.6.2. Java RMI

Java RMI es el middleware de invocación remota que Sun incorporó a la plataforma Java en 1997. No dispone de un IDL ni de compilador de interfaces. La interfaz remota es una interfaz Java convencional, y el *stub* es un *proxy* dinámico que el *runtime* construye mediante reflexión. Eso simplifica mucho su uso, pero cliente y servidor han de estar escritos necesariamente en Java (o, al menos, deben ejecutarse en una JVM). Utiliza su propio protocolo llamado JRMP encapsulado sobre TCP y serializa los datos con la serialización nativa de Java. La semántica de invocación es *at-most-once*.

El ejemplo /`java-rmi` es el «hola mundo» de la documentación oficial de Java. Una interfaz remota (Listado 23.19) debe cumplir dos requisitos : extender *java.rmi.Remote*, una interfaz vacía que solo sirve para marcar el tipo como remoto, y declarar que todos sus métodos pueden lanzar `RemoteException`. Esta es una decisión de diseño deliberada; los fallos de comunicación no se deben ocultar al programador, por eso el compilador obliga al programador a tratarlo en cada invocación.

```
public interface Hello extends Remote {
    String sayHello() throws RemoteException;
}
```

LISTADO 23.19: Interfaz remota *Hello* en Java RMI
/`java-rmi/Hello.java`

El sirviente es una clase que implementa esa interfaz remota declarada (Listado 23.20). El método `exportObject()` de `UnicastRemoteObject` lo registra en el *runtime*, que queda escuchando en el puerto TCP indicado (5000), y devuelve el *stub* de cliente para ese objeto. El servidor Java RMI y el sirviente quedan definidos en este caso en una única clase `Server`.

El *runtime* mantiene activos sus hilos mientras haya objetos exportados, así que el proceso servidor no termina al salir de `main()`.

```
public class Server implements Hello {
    public String sayHello() {
        return "Hello, world!";
    }

    public static void main(String args[]) {
        try {
            Server obj = new Server();
            Hello stub = (Hello) UnicastRemoteObject.exportObject(obj, 5000);

            // Bind the remote object's stub in the registry
            Registry registry = LocateRegistry.getRegistry();
            registry.rebind("Hello", stub);
```

```
            System.err.println("Server ready");

        } catch (Exception e) {
            System.err.println("Server exception: " + e.toString());
            e.printStackTrace();
        }
    }
}
```

LISTADO 23.20: Servidor del ejemplo `Hello` en Java RMI
/java-rmi/Server.java

23.6.2.1. Registry

Para que el cliente pueda obtener ese *stub*, el servidor lo publica en el Registry con el nombre `Hello`. El Registry es un servicio de nombres muy sencillo que se ejecuta con el programa `rmiregistry`[3] y escucha por defecto en el puerto 1099. El propio Registry es un objeto remoto, con identidad fija y conocida. Por eso, `LocateRegistry.getRegistry()` no necesita comunicarse con nada; construye localmente un *stub* a partir del nombre del nodo, el puerto 1099 y esa identidad fija. Así se resuelve el problema de obtener la primera referencia remota. Sin ella no se puede invocar nada.

La invocación `registry.rebind("Hello", stub)` es a su vez una invocación remota sobre el Registry, en la que el *stub* se pasa como argumento. Ese *stub* contiene la dirección IP y el puerto del servidor y la identidad del objeto. La dirección se obtiene resolviendo el nombre del nodo del servidor; si resuelve a una dirección de *loopback*, la referencia no sirve a los clientes de otros nodos. La propiedad `java.rmi.server.hostname` permite indicar la dirección correcta.

El cliente (Listado 23.21) obtiene una copia de ese *stub* con `lookup()`, y a partir de ese momento invoca `sayHello()` directamente sobre el servidor, en el puerto 5000; el Registry no interviene en las invocaciones.

```
public class Client {
    private Client() {}

    public static void main(String[] args) {
        String host = (args.length < 1) ? null : args[0];
        try {
            Registry registry = LocateRegistry.getRegistry(host);
            Hello stub = (Hello) registry.lookup("Hello");
            String response = stub.sayHello();
            System.out.println("response: " + response);
```

[3]El Registry necesita acceder a la clase *Hello* para deserializar el *stub* que recibe, así que `rmiregistry` se debe lanzar desde el directorio que contiene las clases compiladas. Otra opción es que el propio servidor cree el Registry en su proceso con `LocateRegistry.createRegistry()`.

```
        } catch (Exception e) {
            System.err.println("Client exception: " + e.toString());
            e.printStackTrace();
        }
    }
}
```

LISTADO 23.21: Cliente del ejemplo `Hello` en Java RMI
/java-rmi/Client.java

Por seguridad, el Registry solo acepta las operaciones de escritura —`bind()`, `rebind()` y `unbind()`— cuando proceden del mismo nodo. La consulta (`lookup`), en cambio, se puede invocar desde cualquier nodo.

23.6.2.2. Paso por valor y por referencia

Los objetos que implementan *Remote* y han sido exportados se pasan **por referencia**: al serializar un argumento o un resultado, el *runtime* los sustituye por su *stub*, como ocurre con el que el servidor entrega al Registry. El resto se pasan **por valor**, para lo que su clase debe implementar *java.io.Serializable*. Java RMI utiliza la serialización nativa de Java que, a diferencia de XDR o Protocol Buffers, no se limita a tipos declarados en una interfaz: serializa el grafo completo de objetos alcanzables desde el argumento, conservando su estructura.

23.6.2.3. Recolección de basura distribuida

Java RMI incorpora un recolector de basura distribuido que se basa en el conteo de referencias remotas. Cuando un proceso deserializa un *stub*, su *runtime* informa al servidor que aloja el objeto mediante una invocación `dirty()` sobre el DGC de ese servidor (otro objeto remoto con identidad fija), y cuando el recolector local elimina el último *proxy*, lo notifica con `clean()`.

Como el cliente puede terminar sin avisar, cada referencia se concede como un «préstamo» *(lease)* con una duración limitada, 10 minutos por defecto (propiedad `java.rmi.dgc.leaseValue`), que el cliente debe renovar periódicamente. Si el *lease* expira, el servidor decrementa el conteo. Cuando llega a cero, el *runtime* deja de mantener el objeto y el recolector local de la JVM del servidor lo elimina. Si el sirviente implementa la interfaz *Unreferenced*, se le notifica. Como cualquier esquema de conteo de referencias, no es capaz de recolectar ciclos de objetos que se referencian entre sí desde distintos nodos.

En el ejemplo, la única referencia local al sirviente es la variable `obj` de `main()`, que deja de existir cuando esta termina. El objeto sigue vivo porque el Registry mantiene una referencia remota —el *stub*— y renueva su *lease*.

23.6.2.4. Seguridad

JRMP no cifra ni autentica las comunicaciones. Para hacerlo, `exportObject()` admite una variante que recibe las factorías de *sockets* del cliente y del servidor; las clases `SslRMIClientSocketFactory` y `SslRMIServerSocketFactory` permiten usar TLS.

Las primeras versiones de Java RMI permitían que el receptor descargara desde una URL indicada por el emisor *(codebase)* las clases que le faltaban. Las versiones actuales ya no lo permiten, así que ambos extremos necesitan tenerlas en su *classpath*.

Hay otro problema de seguridad más grave. La deserialización de objetos arbitrarios recibidos de la red es un vector de ataque para introducir código malicioso. Un atacante puede enviar un grafo de objetos, construido con clases presentes en el *classpath* del servidor, cuya deserialización acaba ejecutando código arbitrario. Desde Java 9, la plataforma incluye filtros de deserialización que limitan qué clases se aceptan. Aun así, exponer un servicio Java RMI en una red no confiable se considera una mala práctica.

23.6.3. ZeroC Ice

Ice es un middleware RMI desarrollado por la empresa ZeroC, cuya primera versión apareció en 2003. Sus autores procedían del mundo de CORBA y lo concibieron como un sustituto que conservara el modelo de objetos distribuidos, pero con una especificación mas sencilla y un *runtime* más ligero. Genera *stubs* para muchos lenguajes de modo que, igual que en gRPC, cliente y servidor pueden estar escritos en lenguajes distintos. Utiliza un protocolo de aplicación propio llamado IceP, independiente del transporte. Sus mensajes se encapsulan sobre el que indique el *endpoint*.

Las piezas son las habituales de cualquier middleware RMI (§ 23.6): un IDL, un compilador que genera los *stubs* de cliente y servidor, un *runtime* con su adaptador de objetos y una referencia de objeto remoto que el cliente materializa en un proxy.

El ejemplo /`py/counter-by-id` es el servicio `Counter` de pico-rmi (§ 23.6.1) reescrito para Ice: un servidor aloja dos contadores independientes y un cliente invoca sus operaciones.

La interfaz se describe en el lenguaje Slice —el IDL de Ice— (Listado 23.22). Permite declarar módulos, interfaces, estructuras, secuencias, diccionarios, enumerados y excepciones; las interfaces admiten herencia, incluso múltiple.

```
module Example {
    interface Counter {
        long increment();
        idempotent long get();
    };
};
```

Listado 23.22: Interfaz *Counter* en Slice
/py/counter-by-id/counter.ice

Una peculiaridad de Slice es que es posible declarar qué operaciones son idempotentes (`idempotent`). Sabiendo esto, el *runtime* del cliente puede decidir cómo gestionar un fallo. Una operación marcada así se puede reintentar sin romper la semántica *at-most-once* (§ 23.3), algo que no se puede aplicar a un método no idempotente. En `Counter`, `get()` es idempotente e `increment()` no lo es.

Cliente y servidor conocen esa marca porque sus *stubs* se han generado desde la misma interfaz, así que ninguno de los dos necesita que se la indiquen. Aun así, el «modo» de la operación se incluye en cada petición. Está ahí para componentes que no tienen la interfaz, *p. ej.* un intermediario que reenvía invocaciones desconocidas, como el servicio de propagación de eventos IceStorm.

El compilador `slice2py` (`zeroc-ice-compilers`) genera el módulo `counter_ice.py` y un paquete `Example` con el proxy de cliente (`CounterPrx`) y el *stub* de servidor (`Counter`). Pero como Python es un lenguaje dinámico, ese paso se puede evitar. La función `Ice.loadSlice()` compila el fichero `.ice` al vuelo durante la ejecución y construye los tipos en memoria.

El sirviente (`CounterI`) hereda del *stub* de servidor e implementa las dos operaciones (Listado 23.23). Cada método recibe un parámetro que no aparece en la interfaz, `current`, que permite al sirviente acceder a los datos de la propia invocación: la identidad del objeto invocado, el nombre de la operación, el adaptador, o la conexión entre otros. Con ese parámetro, un único sirviente puede encargarse de muchos objetos distintos y distinguirlos por su identidad. Eso es lo que se llama «sirviente por defecto» *(default servant)*, y evita crear un sirviente por cada entidad cuando son miles o millones.

```
class CounterI(Example.Counter):
    def __init__(self):
        self.value = 0

    def increment(self, current):
        self.value += 1
        return self.value

    def get(self, current):
        return self.value

def main(ic):
    adapter = ic.createObjectAdapter('CounterAdapter')

    proxy1 = adapter.add(CounterI(), Ice.stringToIdentity('counter1'))
    proxy2 = adapter.add(CounterI(), Ice.stringToIdentity('counter2'))

    print(proxy1)
    print(proxy2)

    adapter.activate()
    ic.waitForShutdown()
    return 0
```

LISTADO 23.23: Servidor del servicio `Counter` en ICE
/py/counter-by-id/server.py

La función `main()` recibe en `ic` el *comunicador (communicator)*, el objeto principal del *runtime* de ICE, a partir del cual se crean los adaptadores de objetos y los proxies. El programa lo obtiene al arrancar con `Ice.initialize(sys.argv)`, que además extrae de los argumentos de la línea de comandos las propiedades de configuración del *runtime*.

El servidor registra cada sirviente en el adaptador de objetos con `adapter.add()` bajo una identidad única, que se puede construir a partir d euna cadena con `Ice.stringToIdentity()`. Ahí están los dos niveles de despachado: el adaptador localiza el sirviente a partir de la identidad que viene en la petición, y el *stub* del servidor localiza el método a partir del identificador de la operación. Es exactamente el mismo papel del `ObjectAdapter` de pico-rmi, pero el *endpoint* en el que escucha no se le pasa como argumento, sino que lo toma de una propiedad de configuración:

```
CounterAdapter.Endpoints=tcp -h 127.0.0.1 -p 2002
```

Prácticamente todo el comportamiento del *runtime* se ajusta con propiedades como esta, que se leen del fichero indicado en `--Ice.Config` o bien de la línea de comandos. El protocolo de transporte por defecto, el tamaño máximo de mensaje, los plazos de espera, la cantidad de reintentos, el tamaño del *pool* de hilos, las trazas de red y muchas otras características son configurables con propiedades (hay cientos de ellas). El código no cambia

al pasar de TCP a TLS, ni al mover el servidor a otro puerto, solo hay que ajustar las propiedades.

23.6.3.1. Referencias remotas textuales

El método `adapter.add()` devuelve el proxy del objeto recién registrado, e ICE le da una representación textual completa, lo que su documentación llama *stringified proxy*. Eso es precisamente lo que imprime el servidor del ejemplo:

```
counter1 -t -e 1.1:tcp -h 127.0.0.1 -p 2002 -t 60000
```

Esa cadena es una referencia de objeto remoto (§ 23.6) representada como texto, y contiene los dos elementos que la definen: la identidad del objeto (`counter1`), y el *endpoint* del servidor que lo aloja (`tcp -h 127.0.0.1 -p 2002`), después. Entre ambos hay varias opciones de la referencia: `-t` es el modo de invocación (*twoway*), `-e 1.1` la versión del formato de serialización y `-t 60000` el *timeout* de invocación en milisegundos.

En realidad cuando se utiliza `print()` con un proxy, se está invocando `proxyToString()`. Y también existe la función inversa `stringToProxy()`, que construye un proxy que el cliente puede invocar.

Que la referencia remota se pueda transportar como texto resuelve el problema de «la primera referencia», el mismo que Java RMI resuelve con el Registry (§ 23.6.2): el cliente la obtiene por cualquier medio —un fichero de configuración, un argumento en la línea de comandos, un mensaje de correo— sin necesidad de un servicio de nombres. También hace evidente algo que otros middlewares ocultan: una referencia remota puede llevar varios *endpoints*, incluso con protocolos distintos (`tcp`, `ssl`, `udp`), y el *runtime* prueba cada uno de ellos hasta conseguir establecer la comunicación. Si el adaptador del ejemplo no fijara la dirección con `-h`, escucharía en todas las del nodo y la referencia incluiría un *endpoint* por cada una.

23.6.3.2. Reflexión remota

El método `stringToProxy()` solo considera la cadena que se le pasa (el proxy textual); no comprueba que el objeto exista ni que sea del tipo esperado. Devuelve un proxy de tipo `Ice::ObjectPrx`, la clase base de todos los proxies. Para invocar un método de la interfaz declarada —`increment()` en este ejemplo— es necesario que el tipo de este proxy sea del tipo específico (`CounterPrx`). De eso se encarga `checkedCast` (Listado 23.24).

```
def main(ic):
    proxy = ic.stringToProxy(sys.argv[1])
    counter = Example.CounterPrx.checkedCast(proxy)

    if counter is None:
        raise RuntimeError('invalid proxy')

    print("get() = '{}'".format(counter.get()))
    print("increment() = '{}'".format(counter.increment()))
    print("increment() = '{}'".format(counter.increment()))
    print("get() = '{}'".format(counter.get()))
```

LISTADO 23.24: Cliente del servicio `Counter` en ICE
/py/counter-by-id/client.py

Convertir una referencia de un tipo base a uno derivado es un **molde hacia abajo** *(downcasting)*, una operación habitual en cualquier lenguaje orientado a objetos y que no es nada particular de ICE o RMI. Lo que sí cambia es quién puede comprobarlo. En local, el objeto está en memoria y el lenguaje puede hacerlo, pero con RMI el único que sabe de qué tipo es el objeto es el servidor. Por eso, `checkedCast()` en realidad implica una invocación remota que interroga al objeto para verificar su tipo. Hay una variante `uncheckedCast()` que construye el proxy, pero no comprueba, algo que lógicamente implica un riesgo de fallo, pero hay situaciones en las que simplemente no es posible hacer la comprobación.

Preguntarle a un objeto por su propio tipo es un mecanismo de **reflexión**, «remota» en este caso. Toda interfaz SLICE hereda implícitamente de *Ice::Object*, que aporta a cualquier objeto remoto las operaciones necesarias para preguntarle sobre sí mismo. Su alcance, eso sí, es corto: el objeto revela qué interfaces implementa (método `ice_ids`), pero no qué operaciones tiene cada una ni cómo se serializan sus argumentos. ICE no ofrece un repositorio de interfaces que se pueda consultar en ejecución, como sí tenía CORBA.

En el ejemplo `Counter`, cada ejecución del cliente invoca `get()`, dos veces `increment()` y de nuevo `get()` sobre el objeto de la referencia recibida como argumento. Igual que en pico-rmi, los dos contadores llevan cuentas independientes aunque compartan servidor, adaptador e interfaz.

23.6.3.3. Paso de referencias y ciclo de vida

En SLICE, el tipo de un proxy se escribe con un asterisco tras el nombre de la interfaz (`Counter*`), y se puede usar como parámetro o como resultado de una operación. Las referencias de objeto remoto son, por tanto, «ciudadanos de primera clase» (§ 23.6), con los dos modos de paso que eso implica. Los objetos remotos se envían por referencia, y el receptor construye con

Servidor

```
$ ./server.py --Ice.Config=server.config
counter1 -t -e 1.1:tcp -h 127.0.0.1 -p 2002 -t 60000
counter2 -t -e 1.1:tcp -h 127.0.0.1 -p 2002 -t 60000
```

Cliente

```
$ ./client.py 'counter1 -t:tcp -h 127.0.0.1 -p 2002'
get() = '0'
increment() = '1'
increment() = '2'
get() = '2'
$ ./client.py 'counter1 -t:tcp -h 127.0.0.1 -p 2002'
get() = '2'
increment() = '3'
increment() = '4'
get() = '4'
$ ./client.py 'counter2 -t:tcp -h 127.0.0.1 -p 2002'
get() = '0'
increment() = '1'
increment() = '2'
get() = '2'
```

FIGURA 23.4: Servidor y cliente ICE
/py/counter-by-id

ella un proxy cuyas invocaciones llegan al original; el resto de los tipos —números, cadenas, estructuras, secuencias, diccionarios— se envían por valor. La diferencia con Java RMI, que serializa el grafo completo de objetos alcanzables (§ 23.6.2), es que aquí solo se envía lo que declare la interfaz.

Donde ICE se aparta de Java RMI es en el ciclo de vida. ICE no incorpora recolector de basura distribuido. Es responsabilidad del programador. A cambio, el servidor no mantiene conteos de referencias ni *(leases)* que renovar, y un cliente que desaparece sin avisar no deja nada pendiente.

23.6.3.4. Modos de invocación

El modo de invocación no forma parte de la interfaz; es un atributo de la referencia remota, que el cliente puede cambiar en su proxy cuando necesite. Corresponde con el `-t` que aparece en el proxy textual, aunque el objeto `Ice::ObjectPrx` también tiene métodos para lograr lo mismo en ejecución. El modo determina el estilo de intercambio y, por tanto, la semántica de invocación que obtiene el cliente (Cuadro 23.2).

El modo *twoway* (por defecto) es el único que permite obtener respuestas. El cliente recibe el resultado o una excepción, y como TCP evita pérdidas y duplicados, y el *runtime* no reintenta por su cuenta una operación que no sea idempotente, la semántica es *at-most-once* (§ 23.4).

Opción	Modo	Intercambio	Semántica	Método `ObjectPrx`
`-t`	*twoway*	RR sobre TCP	*at-most-once*	`ice_twoway()`
`-o`	*oneway*	R sobre TCP	*maybe*	`ice_oneway()`
`-d`	*datagram*	R sobre UDP	*maybe*	`ice_datagram()`
`-O`	*batch oneway*	R sobre TCP	*maybe*	`ice_batchOneway()`
`-D`	*batch datagram*	R sobre UDP	*maybe*	`ice_batchDatagram()`

CUADRO 23.2: Modos de invocación en una referencia remota ICE

Los otros cuatro cubren solo el mensaje de petición. El *stub* de cliente lo entrega al transporte y devuelve el control inmediatamente, sin esperar. El cliente no puede saber si la operación llegó a ejecutarse. Incluso con TCP, que garantiza que el mensaje llega al extremo remoto, no es posible saber si el sirviente ejecutó correctamente la operación (fallo parcial). Por eso la semántica es *maybe* en los cuatro casos. Lo que los distingue es el coste y el riesgo: con `-d` y `-D` el mensaje puede perderse por el camino, mientras que las variantes «por lotes» *(batch)* no envían nada hasta que se vacía la cola, lo que amortiza la sobrecarga cuando las invocaciones son muchas y pequeñas, a cambio de perderlas todas juntas si hay un problema en la conexión.

Por coherencia, un proxy de solo petición únicamente admite operaciones sin resultado ni excepciones declaradas; invocar `get()` en esos casos produce un error local (`TwowayOnlyException`) antes de enviar nada. La misma interfaz sirve para los dos estilos de intercambio (§ 23.1) sin cambiar la definición, algo que por ejemplo en gRPC exigiría declararlo en el fichero `.proto`.

Existe una opción más en los proxies textuales: `-s`, que no es un modo sino un requisito: obliga al *runtime* a usar únicamente *endpoints* seguros, y por eso aparece combinada con la del modo (`-t -s`).

23.7. Localización de servicios

De uno u otro modo, cualquier referencia de objeto remoto incluye el *endpoint* del servidor que aloja al objeto (§ 23.6). Eso resuelve el problema de encontrarlo, pero crea otro: acopla el cliente a un `c`:oncreto. Así lo hacen todos los ejemplos de este capítulo: pico-rmi recibe el host como argumento, gRPC construye su canal con un *endpoint* fijo, el proxy de ICE lo lleva incluido, etc. Esto funciona mientras nada cambie, pero en un despliegue las cosas cambian. Los servidores cambian de nodo (se «migran»), se replican, o se reinician en otro puerto. También es útil poder arrancar los servidores solo cuando hacen falta («bajo demanda»).

La **transparencia de localización** ofrece a los clientes nombres alternativos para los servidores que no incluyen endpoints. Para lograr la comunicación obviamente los endpoints son necesarios, pero se puede aplazar el momento de averiguarlos hasta que se produzca la invocación. Para lograr esto, el middleware debe incluir un servicio que proporcione los endpoints —potencialmente efímeros— para los identicadores estables de los servicios. El componente que hace esto es el *binder*.

Todo *binder* tiene el problema del *bootstrapping*, es decir, ¿cómo se obtienen sus endpoints, si para eso hace falta precisamente el servicio que ofrece el *binder*? Es el mismo problema de los servidores DNS, y la solución también suele ser la misma. El *binder* (al igual que el servidor DNS) tiene un *endpoint* fijo, conocido y estable, ya sea por convención o por configuración. El *binder* de ONC-RPC es `rpcbind` (§ 23.5.2), que escucha siempre en el puerto 111, y el de Java RMI es el Registry (§ 23.6.2), en el 1099 y con una identidad fija. En ICE ese papel lo realiza un servicio llamado `Locator`, cuyo proxy se fija con la propiedad `Ice.Default.Locator`, que tiene este aspecto:

```
Ice.Default.Locator=IceGrid/Locator:tcp -h maine -p 4061
```

La alternativa al *binder* es el «descubrimiento de servicios». En lugar de preguntarle a un servicio conocido, se envía una consulta a la red con algún mecanismo de difusión —`rpcbind` e ICE lo tienen. El problema es que la difusión normalmente está localizada únicamente a la LAN o la subred.

23.7.1. Registro de servidores y objetos

Para que el *binder* pueda hacer su trabajo, los servidores tienen registrarse en él. En ONC-RPC el servidor registra en `rpcbind` cada combinación de programa, versión y protocolo junto al puerto que el sistema le ha asignado. La tabla resultante se consulta con `rpcinfo -p`. Con el servidor del ejemplo /`onc-rpc-hello` en marcha, sus dos últimas entradas corresponden al programa `600000000` que declara su interfaz, registrado por partida doble porque acepta los dos transportes:

```
$ rpcinfo -p
   program vers proto   port  service
    100000    4   tcp    111  portmapper
    100000    3   tcp    111  portmapper
    100000    2   tcp    111  portmapper
    100000    4   udp    111  portmapper
    100000    3   udp    111  portmapper
    100000    2   udp    111  portmapper
 600000000    1   udp  57269
 600000000    1   tcp  62357
```

Los puertos son aleatorios y cambian en cada arranque; eso es justamente por lo que hace falta un *binder*, y lo que permite por ejemplo que un servidor NFS no necesite un puerto fijo. Obsérva que `rpcbind` figura en su propia tabla (`100000 «portmapper`). El binder es un programa RPC más, igual que el Registry de Java RMI es un objeto remoto Java y el localizador de Ice un objeto Ice.

En Java RMI el registro es explícito y lo hace el programador con `rebind()`, que publica el *stub* bajo un nombre. El Registry *stub* completo, no solo el endpoint.

Ice lo automatiza por completo. El adaptador de objetos, al activarse, registra por su cuenta en el *binder* el «proxy directo» *(direct proxy)* —el que lleva *endpoints*, como los de § 23.6.3— en el que va a escuchar; el programador no tiene que escribir nada de código, solo le da al adaptador un identificador y configura un `Locator`, ambos por configuración (Listado 23.25). Obsérva que el adaptador ni siquiera fija el puerto (`tcp`), porque no hace falta: el *runtime* elige uno libre y es el *binder* quien lo almacena.

```
PrinterAdapter.Endpoints=tcp
PrinterAdapter.AdapterId=PrinterAdapter1
PrinterAdapter.Locator=IceGrid/Locator -t:tcp -h 127.0.0.1 -p 4061
```

Listado 23.25: Configuración del servidor con localizador
/locator/server.config

El servidor es exactamente el mismo del «hola mundo» (/py/hello/server.py). Lo único que cambia es lo que imprime: el proxy ya no lleva *endpoints*, sino el nombre del adaptador tras una `@`. Es un «proxy indirecto» *(indirect proxy)*.

```
$ ./server.py --Ice.Config=../locator/server.config
printer1 -t -e 1.1 @ PrinterAdapter1
```

23.7.2. Resolución de nombres

La resolución es transparente para el programador. En Java RMI la consulta al Registry es una línea de código que alguien tiene que escribir (`lookup()`), y ocurre una vez, antes de tener el proxy. En ONC-RPC, `clnt_create()` hace lo propio contra `rpcbind`. En Ice no hay ninguna llamada equivalente; el cliente construye el proxy y lo invoca, y es el *runtime* el que, al intentar establecer la comunicación, observa que la referencia no tiene *endpoints* y consulta al localizador antes de conectar. El cliente solo necesita saber dónde está el localizador (Listado 23.26) y usar como referencia la que imprimió el servidor. Los *endpoints* que el *runtime* obtiene quedan en una caché para evitar resoluciones adicionales innecesarias.

```
Ice.Default.Locator=IceGrid/Locator -t:tcp -h 127.0.0.1 -p 4061
```

Listado 23.26: Configuración del cliente con localizador
/locator/locator.config

```
$ ./client.py --Ice.Config=locator.config "printer1 -t -e 1.1 @ PrinterAdapter1"
```

Esa caché caduca por fallo, no por tiempo. Cuando el *runtime* no consigue conectar con los *endpoints* cacheados, descarta la entrada, vuelve a preguntarle al localizador y reintenta la invocación. Esa *re-resolución* es lo que da transparencia de verdad. Permite mover un servidor de puerto o de nodo mientras un cliente lo está usando, sin que reciba ningún error. Aunque solo oculta los fallos de conexión; si la petición ya salió y la operación no es `idempotent`, reintentar rompería la semántica *at-most-once* y el error sí llega al cliente.

La diferencia con los otros dos binders es evidente. El *stub* que devuelve el Registry de Java RMI lleva el *endpoint* y no se renueva nunca. Si el servidor cambia de puerto, ese *stub* queda inservible y el programador es quien tiene que repetir la llamada a `lookup()`. Lo mismo ocurre con `clnt_create()` en ONC-RPC, que consulta a `rpcbind` al crear el manejador y no vuelve a hacerlo. En ambos casos el *binder* resuelve el arranque; en Ice, además, vigila la validez de la referencia durante toda su vida. Cap'n Proto adopta un enfoque distinto, que renuncia al binder por diseño hasta el punto de considerarlo una importante debilidad (§ 23.9).

23.8. Invocación y despachado asíncrono

Las invocaciones que se han tratado hasta ahora son «síncronas», es decir, el hilo del cliente queda bloqueado desde que envía la petición hasta que llega la respuesta. Se hace así como analogía a la forma en que funcionan las llamadas locales convencionales. Sin embargo, en una invocación remota ese tiempo de espera —RTT más lo que tarde el servidor en ejecutar la operación— se pierde. Hay dos mecanismos para aprovecharlo, uno en cada extremo, y son independientes porque resuelven esperas distintas. La *invocación asíncrona*, de AMI (Asynchronous Method Invocation), permite al cliente seguir ejecutando código en lugar de esperar, y el *despachado asíncrono*, AMD (Asynchronous Method Dispatching), evita que se bloquee el servidor. Ninguno de los extremos puede saber cómo lo está haciendo el otro, porque el intercambio de mensajes es el mismo RR de siempre. Una invocación asíncrona sigue esperando una respuesta, la diferencia es que

el programa no se detiene a esperarla. No se debe confundir la invocación asíncrona con la invocación de solo petición, que no tiene respuesta.

Los ejemplos de esta sección utilizan Ice, que es el middleware más completo de los tratados en este aspecto.

23.8.1. Invocación asíncrona

En Ice la decisión es del cliente y no afecta a la interfaz. Para la operación `factorial()` de la interfaz *Math*, declarada como `long factorial(int value)`, el compilador genera siempre dos formas de invocarla en el proxy: `factorial()`, que bloquea, y `factorialAsync()`, que devuelve inmediatamente un *future*. Un *future* representa un resultado que todavía no está disponible.

El programa puede recoger el resultado de varias formas:

- Esperando más tarde: el cliente sigue con su trabajo y bloquea solo cuando necesita el valor. La llamada `result()` espera si la respuesta aún no ha llegado (Listado 23.27).
- Con un *callback*: el cliente registra una función que el *runtime* invoca cuando el *future* se completa, y no espera en ningún momento (Listado 23.28).
- En un bucle de eventos: `Ice.wrap_future()` convierte el *future* en uno de `asyncio`, de modo que se puede usar `await` (Listado 23.29).
- Consultando periódicamente si ya ha llegado el resultado *(polling)*, con `done()` (Listado 23.30).

```
future = math.factorialAsync(value)
print("That was an async call")
print(f"Async result is: {future.result()}")
```

Listado 23.27: Cliente que espera el resultado más tarde
/py/ami/client-block.py

```
def callback(future):
    try:
        print(f"Callback: value is: {future.result()}")
    except Exception as ex:
        print(f"Exception is: {ex}")

def main(ic):
    if len(sys.argv) != 3:
        print(f"usage: {sys.argv[0]} <proxy> <value>")
        return 1
```

```
    proxy = ic.stringToProxy(sys.argv[1])
    math = Example.MathPrx.checkedCast(proxy)
    value = int(sys.argv[2])

    if not math:
        raise RuntimeError('Invalid proxy')

    future = math.factorialAsync(value)
    future.add_done_callback(callback)

    print("That was an async call")

    return 0
```

Listado 23.28: Cliente con *callback*
/py/ami/client-callback.py

```
result = await Ice.wrap_future(math.factorialAsync(value))
print(f"Async result is: {result}")
```

Listado 23.29: Cliente integrado en un bucle de eventos
/py/ami/client-await.py

```
future = math.factorialAsync(value)
while not future.done():
    print("Still waiting...")
    time.sleep(0.1)
print(f"Async result is: {future.result()}")
```

Listado 23.30: Cliente que consulta periódicamente
/py/ami/client-poll.py

Que la invocación no bloquee tiene consecuencias. La excepción no se puede lanzar en el punto de la llamada, que ya ha terminado, sino que incluye en el *future* y se eleva al pedir el resultado; por eso el *callback* del ejemplo captura la excepción al leerlo. Además, el *callback* normalmente se ejecuta en un hilo del *runtime*, no en el que hizo la invocación, así que si comparte datos con el resto del programa hay que sincronizar el acceso. Y la semántica de invocación no cambia; una invocación asíncrona sigue siendo *at-most-once*, igual que la síncrona. Cambia cuándo la recibe el cliente, no qué garantías tiene.

23.8.2. Despachado asíncrono

En el servidor el problema es otro. El *runtime* atiende las peticiones con un *pool* de hilos limitado —por omisión, de un solo hilo, ajustable con la propiedad `Ice.ThreadPool.Server.Size`—, y el hilo que recibe una petición ejecuta el sirviente y bloquea hasta que retorna. Si la operación es lenta, los hilos del pool quedan ocupados. Unas pocas peticiones simultaneas pueden

agotar el *pool* e impedir que se atiendan las demás. El despachado asíncrono desacopla la ejecución de la operación del envío de la respuesta. Con AMD el sirviente devuelve el control al *runtime* sin tener el resultado, y este envía la respuesta más tarde, cuando el resultado esté disponible.

A diferencia de la invocación asíncrona, esta decisión sí aparece en la interfaz, porque cambia la firma que tiene el sirviente. En el Slice se declara con una anotación en la operación, `[.ªmd"] long factorial(int value)`. El sirviente del ejemplo (Listado 23.31) no calcula nada; crea un *future*, lo entrega con el argumento a una cola de trabajo que atiende otro hilo, y devuelve un *future*. La respuesta no sale al retornar el método, sino cuando se complete ese *future*. En el ejemplo lo hace el *job* al ejecutarse (Listado 23.32), con `set_result()`, o con `set_exception()` si tiene que cancelarse, para que el cliente reciba un error en lugar de esperar indefinidamente.

```
class MathI(Example.Math):
    def __init__(self, work_queue):
        self.work_queue = work_queue

    def factorial(self, value, current=None):
        future = Ice.Future()
        self.work_queue.add(future, value)
        return future
```

Listado 23.31: Sirviente con despachado asíncrono
/py/amd/server.py

```
class Job:
    def __init__(self, future, value):
        self.future = future
        self.value = value

    def execute(self):
        self.future.set_result(factorial(self.value))

    def cancel(self):
        self.future.set_exception(Example.RequestCanceledException())
```

Listado 23.32: Tarea *(job)* que completa el *future*
/py/amd/work_queue.py

El cliente que se utiliza con este servidor AMD es uno síncrono corriente, que invoca `factorial()` y espera el resultado. Este mecanismo no hace que el cálculo sea más rápido. El factorial es cálculo puro y la cola de trabajo no le aporta nada. La ventaja aparece cuando la operación espera, *p. ej.* una consulta a una base de datos o a otro servicio remoto, porque mientras espera no bloquea ningún hilo del *runtime*.

23.9. Cap'n Proto

La arquitectura de Cap'n Proto es la de cualquier RMI (§ 23.6): un IDL, un compilador, *stubs* y un *runtime*, con un modelo de objetos distribuidos. Sin embargo, su diseño cuestiona y, de hecho va contra, algunos de los pilares que rigen la mayoría de los otros middlewares: la necesidad de serializar los datos, el modelo de referencias remotas y la forma de obtenerlas.

Las siguientes secciones examinan cada una de ellas sobre el mismo ejemplo (/`capn-upper`), un servicio *TextProcessor* con una única operación, `upper()`, que devuelve una versión en mayúsculas del texto recibido. Existe en C++ y en Python, con el mismo fichero de interfaz (Listado 23.33). El fichero comienza con un identificador de 64 bits que lo identifica de forma inequívoca, y cada método lleva un número de orden (`@0`) que cumple el mismo papel que el número de campo de Protocol Buffers, es decir, permite ampliar la interfaz con métodos nuevos sin romper la compatibilidad.

```
@0x8b96e1f756a41043;

interface TextProcessor {
  upper @0 (message :Text) -> (result :Text);
}
```

Listado 23.33: Interfaz *TextProcessor* en Cap'n Proto
/`capn-upper/py/upper.capnp`

El compilador `capnp` (`capnproto`) genera el código de los *stubs* para C++. En Python no hace falta ese paso: el módulo `capnp` carga el fichero de interfaz directamente al importar `upper_capnp`, de un modo similar a `Ice.loadSlice()`.

23.9.1. ¿Sin serialización?

Enviar un argumento con XDR, Protocol Buffers o la serialización de Java implica convertirlo desde su representación en memoria a una secuencia de bytes, y el receptor debe hacer la conversión inversa. Cap'n Proto elimina ese paso: el mensaje se construye directamente en un *buffer* con la misma disposición que tendrá en el cable, y el receptor lo lee sin transformarlo. Esto es lo que se llama serialización *zero-copy*.

Para que esto sea viable, los tipos que genera el compilador no son objetos cuyos campos se rellenan al deserializar, sino accesores que operan sobre el *buffer*. En el cliente del ejemplo (Listado 23.35), el argumento `message` de `upper()` escribe el texto directamente en el mensaje que se va a enviar, y el resultado (`reply.result`) se lee del *buffer* tal como llegó.

Un mensaje que no necesita interpretarse para leerlo se puede leer en parte sin recorrer el resto, procesar antes de haberlo recibido completo, o guardar en un fichero y proyectar en memoria sin releerlo. Como contrapartida, ocupa más que uno con codificación compacta.

23.9.2. Referencias como *capabilities*

En Cap'n Proto las referencias a objetos remotos se llaman *capacidades (capabilities)*. Son la referencia de objeto remoto de § 23.6, pero con una propiedad añadida: designan un objeto y, además, le dan el permiso para invocarlo. En Java RMI y en Ice una referencia es un dato, una identidad más un *endpoint*, y con eso basta para poder invocar el objeto. En Cap'n Proto una capacidad no se puede fabricar ni se puede delegar a un tercero. Cuando un nodo envía una referencia a otro, el identificador únicamente es válido para esa conexión. Nadie que no lo haya recibido directamente puede invocar.

Colocar la referencia y el permiso juntos no es un mero detalle de implementación; es la respuesta a un problema de seguridad. Cuando una referencia se puede conseguir sin que nadie la haya concedido, el permiso hay que comprobarlo en otro lugar. El servidor debe averiguar quién es el cliente e implementar un mecanismos de autorización.

Ese enfoque conlleva dos inconvenientes:

- Cada cliente tiene permiso para acceder a todo aquello que pueda nombrar, y no solo de lo que necesita para su tarea. Viola el el principio de mínimo privilegio.
- El problema del «diputado confundido» *(confused deputy)*: un servicio con autoridad que actúa en nombre de un cliente, sobre un recurso cuyo nombre le indica el cliente, puede acabar usando esa autoridad sobre un recurso distinto para el que el cliente no tiene permiso.

Con capacidades, el cliente no le indica al servicio un nombre, sino que le entrega la referencia, que designa el recurso y prueba a la vez que tiene permiso para usarlo. La política de acceso deja de ser una tabla que mantener y pasa a ser una decisión de diseño de las interfaces. Es la misma idea que se aplica en un sistema de ficheros en un SO; para conceder a un usuario acceso de solo lectura a un fichero, se le entrega un objeto que no permite escribir —puede que ni exista la operación de escritura— y no hay nada que el cliente pueda hacer.

Esta idea también cambia la forma de obtener la primera referencia. No hay Registry, servicio de nombres ni referencia con identidad conocida. Al

establecer la conexión, cada extremo puede ofrecer un único objeto inicial, el *bootstrap*, que el otro obtiene como primera capacidad. En el servidor del ejemplo (Listado 23.34), es el argumento `bootstrap` de `TwoPartyServer`, y el cliente lo recoge con el método `bootstrap()` (Listado 23.35). La conexión es a la vez el canal y el nombre.

```
class TextProcessorI(upper_capnp.TextProcessor.Server):
    async def upper(self, message, _context):
        print("Client sent:", message)
        return message.upper()

async def new_connection(stream):
    await capnp.TwoPartyServer(stream, bootstrap=TextProcessorI()).on_disconnect()
```

LISTADO 23.34: Servidor de *TextProcessor* en Cap'n Proto
/capn-upper/py/server.py

```
async def main(host, port):
    connection = await capnp.AsyncIoStream.create_connection(
        host=host, port=port)

    client = capnp.TwoPartyClient(connection)
    text_processor = client.bootstrap().cast_as(upper_capnp.TextProcessor)

    reply = await text_processor.upper(message="hello, world!")
    print("Server reply:", reply.result)
```

LISTADO 23.35: Cliente de *TextProcessor* en Cap'n Proto
/capn-upper/py/client.py

La ausencia de un servicio de nombres es deliberada, no una carencia. Un servicio de nombres es justamente una fuente de referencias que los clientes pueden conseguir sin que nadie les haya concedido permiso expreso para usarlas, como el *Registry* de Java RMI o el *Locator* de ICE. Con Cap'n proto, los objetos se alcanzan navegando desde el objeto inicial, que entrega a cada cliente solo lo que debe ver; si una aplicación necesita algo parecido a un servicio de nombres, lo implementa como un objeto más. Esto no resuelve el problema de localización de § 23.7; el cliente sigue necesitando el *endpoint* del servidor al que se conecta.

Las capacidades están ligadas a la conexión, lo que simplifica el ciclo de vida sin necesidad de *leases*. Un objeto se destruye cuando se pierden todas sus referencias o la conexión por la que se recibieron. En el ejemplo, el servidor crea un sirviente por conexión (`new_connection()` en el listado), que desaparece con ella y obliga al cliente a reconectar. El protocolo se transporta sobre un flujo de bytes, como TCP en el ejemplo.

Este modelo tiene inconvenientes. Revocar una capacidad ya entregada es difícil, y responder a quién tiene acceso a un objeto exige reconstruir el grafo de referencias repartidas, sin una tabla central que consultar. Una caída de conexión invalida todas sus capacidades, y el cliente debe reconectar y reconstruir los objetos que usaba. Para guardar una capacidad y restaurarla más tarde en otra conexión hay que convertirla en un *token* persistente, una parte opcional del protocolo, y no todas las capacidades se pueden guardar. Por último, sin servicio de nombres, el descubrimiento queda fuera del middleware.

23.9.3. Promesas y *pipelining*

Cap'n Proto no intenta que una invocación remota se parezca a una local. Toda invocación devuelve inmediatamente una promesa *(promise)*[4], un resultado que todavía no está disponible. El cliente decide con `await` cuándo esperarlo. El diseño de Cap'n Proto pone mucho incapié es no ocultar la red y sus limitaciones, y para lograrlo, la promesa es esencial para aplicar optimizaciones.

Si las referencias son ciudadanos de primera clase, un patrón habitual es invocar una operación que devuelve un objeto e invocar de inmediato otra sobre ese resultado. Con un servicio de ficheros, cuyo método `open()` devuelve un objeto *Directory* o *File*, la lectura de un fichero requiere tres invocaciones encadenadas, aquí en pseudocódigo:

```
directory = root.open("bar")
file = directory.open("foo")
data = file.read()
```

En cualquier otro middleware, cada una tiene que esperar a que llegue la respuesta de la anterior para poder usar su resultado, lo que cuesta tres viajes de ida y vuelta. En Cap'n Proto, la promesa que devuelve `open()` ya ofrece los métodos del objeto que devolverá, y las invocaciones sobre ella se envían al servidor sin esperar: el cliente cursa las tres peticiones juntas, cada una con una referencia al resultado de la anterior, y el servidor las va ejecutando en cadena. Es lo que se llama *promise pipelining*, y reduce la secuencia a un único viaje de ida y vuelta. Su documentación lo presenta como un «viaje en el tiempo»: el cliente recibe la promesa, y puede usarla como destino de nuevas invocaciones, antes de que el servidor haya recibido siquiera la primera petición. Solo funciona cuando todos los objetos de la cadena los aloja el mismo servidor.

[4] El mismo concepto que el *future* de § 23.8

Esto ataca directamente el coste que § 23.6 atribuía al modelo de objetos: la facilidad para repartir referencias invita a interfaces con muchas invocaciones pequeñas, y la latencia acumulada las penaliza. Es la pieza que hace viable la ausencia de nombres: si todo se alcanza navegando desde el objeto inicial, cada paso de la navegación costaría un viaje de ida y vuelta, y sin *pipelining* la tentación sería sustituirlos por rutas y nombres globales. Con *pipelining* no hace falta sacrificar la interfaz, fusionando operaciones para ahorrar viajes —*p. ej.* un método que reciba la ruta completa—, ni la ventaja de poder entregar a cada cliente solo el objeto al que debe llegar. El ejemplo, con una única invocación, no lo necesita.

23.10. REST

REST no es un middleware como los anteriores, es un *estilo arquitectural*, es decir, un conjunto de normas y restricciones que debe cumplir un sistema distribuido [68].

La abstracción central de REST es el *recurso*. Cualquier información que se pueda nombrar puede ser un recurso, ya sea un documento, una imagen, una colección de otros recursos o incluso un servicio cuyo valor cambia con el tiempo, como «el tiempo que hace hoy en Brazatortas».

Estas son las restricciones:

1. Cliente-servidor.
2. El servidor no tiene estado.
3. Las respuestas se pueden cachear.
4. La interfaz es uniforme, es decir, la misma para cualquier recurso.
5. Permite intermediarios entre cliente y servidor.

Hay una sexta restricción (aunque opcional); la de *código bajo demanda*, que permite al servidor enviar código ejecutable al cliente.

Sin embargo, REST no impone ningún protocolo ni formato de serialización específico. Aunque HTTP es, con mucha diferencia, el protocolo más utilizado para implementar REST, en realidad se podría utilizar cualquiera que encaje con esas restricciones. Por ejemplo, CoAP [69] sigue el mismo modelo sobre UDP para dispositivos con recursos muy limitados. Con la serialización pasa lo mismo. JSON es lo habitual, pero un recurso se puede representar en cualquier formato que entiendan ambos extremos, como podría ser Protocol Buffers, XML o incluso texto plano, y no dejaría de ser REST.

Al igual que en JSON-RPC (§ 23.5.4), no hay IDL, compilador, *stubs* ni *runtime* específico. Si el protocolo elegido es HTTP, se puede utilizar cual-

quier cliente desde `curl` hasta un navegador, y lo mismo se puede decir del servidor.

Aunque gRPC también se encapsula en HTTP (§ 23.5.3), lo usa de un modo muy distinto. Para gRPC, HTTP es solo el transporte. Todas sus invocaciones son `POST` y la operación va en la ruta (`/math.Math/factorial`). En REST, en cambio, la ruta identifica un recurso y el método HTTP es la operación. Para REST, HTTP es el protocolo de aplicación, no de transporte.

23.10.1. Quizzy

El resto de la sección se apoya en /`quizzy`, una pequeñísima aplicación de cuestionarios de elección múltiple escrita específicamente para ilustrar los conceptos de REST. Permite crear preguntas, agruparlas en cuestionarios y responder un cuestionario para obtener una calificación. Por simplicidad, no hay usuarios y las calificaciones no se almacenan.

Quizzy maneja tres tipos de datos:

- Una pregunta tiene un enunciado, varias opciones y el índice de la opción correcta.
- Un cuestionario tiene un título y una lista de preguntas. Una misma pregunta puede formar parte de varios cuestionarios.
- Una calificación indica los puntos obtenidos sobre el total de preguntas del cuestionario. Cada respuesta correcta vale un punto.

Las preguntas y los cuestionarios son los recursos del servicio, y cada uno tiene su propia URI. Lo habitual es nombrar cada colección en plural (`/questions`, `/quizzes`) e identificar cada elemento añadiendo su identificador a la ruta de la colección (`/questions/1`). Para crear una pregunta, el cliente envía un `POST` a la colección. El servidor le asigna un identificador y responde con el código 201 (*Created*) y una cabecera `Location` con la ruta del nuevo recurso.

La calificación es distinta, porque no se almacena. La ruta `/quizzes/1/grade` no identifica nada que se pueda consultar después, sino un recurso cuya única función es procesar las respuestas que recibe.

Todo el servidor está en un único fichero, `server.py`, que tiene tres partes: el modelo de datos del ORM, la comprobación de esquema y los manejadores de cada operación.

Se apoya en varios módulos:

FastAPI (`python3-fastapi`)
: Es un *framework* web. Asocia cada combinación de método y ruta con una función Python, el *manejador*, mediante un decorador. Con ayuda de Pydan-

tic, convierte el JSON de cada petición en objetos Python, y lo que devuelve el manejador en una respuesta. También genera la descripción del API.

uvicorn (`python3-uvicorn`)
: Es el servidor HTTP. Acepta las conexiones TCP, interpreta las peticiones y se las entrega a FastAPI.

Pydantic (`python3-pydantic`)
: Permite definir los *esquemas*, clases que describen el formato de los datos recibidos y devueltos, y comprueba que se cumple.

SQLAlchemy (`python3-sqlalchemy`)
: Es el ORM. Asocia las clases `Question` y `Quiz` con tablas de la base de datos, y traduce las operaciones sobre esos objetos a consultas SQL.

SQLite
: Es la base de datos. Se guarda en un único fichero (`quizzy.db`). El módulo que la maneja (`sqlite3`) forma parte de la librería estándar de Python.

En el manejador que crea una pregunta (Listado 23.36), el decorador lo asocia con `POST` sobre `/questions` y fija el código de estado 201. FastAPI le entrega los datos ya validados y convierte en JSON el objeto que devuelve, así que lo único que hace el manejador con HTTP es añadir la cabecera `Location`.

```
@app.post('/questions', status_code=201)
def create_question(data: QuestionData, response: Response,
                    session: SessionDep) -> QuestionWithSolution:
    question = Question(**data.model_dump())
    session.add(question)
    session.commit()
    response.headers['Location'] = f'/questions/{question.id}'
    return question
```

LISTADO 23.36: Manejador que crea una pregunta
/quizzy/server.py

El programa `client.py` no depende de ninguna de ellas. Utiliza `requests` (`python3-requests`), una biblioteca HTTP de propósito general, para crear dos preguntas y un cuestionario, consultarlo y calificarlo. Las mismas peticiones se pueden hacer con `curl`, como se muestra en /quizzy/README.md. Esto demuestra que el servidor ofrece una interfaz que se puede invocar desde clientes distintos, escritos en lenguajes distintos, sin necesidad de *stubs* ni de un *runtime*.

23.10.2. Recursos y representaciones

En Quizzy, cada pregunta y cada cuestionario es un recurso. Su nombre se mapea a una URI, que tiene una función similar a la referencia textual de

ICE (§ 23.6.3). Contiene el *endpoint* (`http://host:puerto`) y la identidad del recurso (la ruta).

> Es muy habitual ver el término «endpoint» para designar una ruta, o una combinación de método y ruta (*p. ej.* `PUT /quizzes/1`). Es un uso de la jerga que no coincide con la terminología de la tesis de Fielding [68], el documento fundacional de REST, que habla de recursos y de sus identificadores (URI). En nuestro caso, *endpoint* designa siempre la tupla dirección-puerto-protocolo, como en el resto del libro.

El cliente REST no obtiene el recurso, ni un proxy para invocar. El cliente REST solo maneja representaciones del estado de los recursos. Por eso, su nombre lo define perfectamente: transferencia de representación de estado. Sin embargo, un mismo recurso puede tener varias representaciones. En Quizzy, una pregunta incluye la respuesta correcta cuando se pide directamente, pero no cuando forma parte de un cuestionario.

En el servidor, la misma pregunta existe de tres formas: una fila en una tabla de la base de datos, un objeto Python y su representación JSON. El ORM convierte entre las dos primeras, y Pydantic entre las dos últimas.

El modelo del ORM (Listado 23.37) define una clase por cada tabla, cuyos atributos corresponden a las columnas. Por simplicidad, las opciones de una pregunta se guardan en una única columna de tipo JSON, en lugar de en una tabla de opciones con clave ajena a la de preguntas. Como una misma pregunta puede formar parte de varios cuestionarios, la relación entre ambos requiere una tabla intermedia sin clase propia (`quiz_questions`). El atributo `questions` de `Quiz` no corresponde a ninguna columna sino a esa relación, y el ORM lo resuelve devolviendo la lista de objetos `Question`.

```
quiz_questions = Table(
    'quiz_questions', Base.metadata,
    Column('quiz_id', ForeignKey('quizzes.id'), primary_key=True),
    Column('question_id', ForeignKey('questions.id'), primary_key=True),
)

class Question(Base):
    __tablename__ = 'questions'
    id: Mapped[int] = mapped_column(primary_key=True)
    text: Mapped[str]
    options: Mapped[list[str]] = mapped_column(JSON)
    correct_option: Mapped[int]

class Quiz(Base):
    __tablename__ = 'quizzes'
    id: Mapped[int] = mapped_column(primary_key=True)
    title: Mapped[str]
```

```
    questions: Mapped[list[Question]] = relationship(
        secondary=quiz_questions, order_by=Question.id)
```

LISTADO 23.37: Modelo de datos del ORM
/quizzy/server.py

Cada representación es un esquema de Pydantic (Listado 23.38). La de un cuestionario incluye sus preguntas como `QuestionWithoutSolution`, que no tiene el campo `correct_option`.

```
class QuestionWithSolution(QuestionData):
    id: int

class QuestionWithoutSolution(BaseModel):
    id: int
    text: str
    options: list[str]
```

LISTADO 23.38: Representaciones de una pregunta
/quizzy/server.py

23.10.3. Interfaz uniforme

En gRPC o en ICE, cada interfaz define sus propias operaciones. En REST, todos los recursos comparten el mismo conjunto (reducido) de operaciones, de modo que el vocabulario del dominio de aplicación se limita a los nombres de los recursos. Si se usa HTTP, esas operaciones son sus mensajes (o «métodos») [70] (Cuadro 23.3).

Método	Seguro	Idempotente	Uso habitual
`GET`	sí	sí	obtener una representación
`PUT`	no	sí	crear o reemplazar un recurso
`DELETE`	no	sí	eliminar un recurso
`POST`	no	no	crear un recurso o procesar datos
`PATCH`	no	no	modificar parcialmente un recurso
`QUERY`	sí	sí	consultar con datos en el cuerpo

CUADRO 23.3: Propiedades REST de los métodos HTTP más habituales

Un método *seguro* es uno que no modifica el estado del servidor. La semántica de cada método forma parte del protocolo, así que cualquier intermediario (un proxy, una caché, el propio navegador) sabe si puede repetir una petición o guardar su respuesta, sin conocer la aplicación. Es la misma información que lleva cada petición ICE sobre las operaciones idempotentes, pero aquí no depende de cada interfaz.

El Cuadro 23.4 muestra la interfaz completa de Quizzy, donde `{id}` representa el identificador de una pregunta o de un cuestionario. Con solo tres métodos se manejan todos sus recursos.

Método	Ruta	Acción
`POST`	`/questions`	crea una pregunta
`GET`	`/questions`	devuelve la lista de preguntas
`GET`	`/questions/{id}`	devuelve una pregunta y su solución
`PUT`	`/questions/{id}`	modifica una pregunta
`POST`	`/quizzes`	crea un cuestionario
`GET`	`/quizzes`	devuelve la lista de cuestionarios
`GET`	`/quizzes/{id}`	devuelve un cuestionario, sin soluciones
`PUT`	`/quizzes/{id}`	modifica un cuestionario
`POST`	`/quizzes/{id}/grade`	califica las respuestas enviadas

CUADRO 23.4: Recursos y operaciones de Quizzy

Quizzy modifica las preguntas y los cuestionarios con `PUT`. El cliente envía la pregunta o el cuestionario completos, y el servidor sustituye el estado anterior por el que recibe. Por eso la operación es idempotente, porque repetirla deja el recurso en el mismo estado. Es posible hacer una actualización parcial de un recurso con `PATCH`, pero no se utiliza en Quizzy.

Calificar un cuestionario no encaja bien en este esquema. No modifica nada en el servidor, así que sería una operación segura e idempotente, pero necesita enviar las respuestas en el cuerpo, y en una petición `GET` el cuerpo no tiene una semántica definida. Quizzy, como muchas aplicaciones, utiliza `POST`, que no garantiza ninguna de esas dos propiedades. El método `QUERY` [71], que acaba de entrar en el estándar de HTTP en 2026, resuelve este problema: es seguro e idempotente, y permite enviar datos en el cuerpo de la petición, pero es demasiado reciente y la mayoría de clientes, servidores, librerías y frameworks no lo soportan aún.

La interfaz uniforme incluye otra restricción que casi ninguna API cumple. Las representaciones deberían incluir enlaces a los recursos relacionados, de modo que el cliente pueda navegar por ellos sin conocer de antemano las URI, igual que una persona navega por la web. Esto se conoce como Hypermedia As The Engine Of Application State (HATEOAS).

23.10.4. Sin estado

Cada petición debe incluir todo lo que el servidor necesita para atenderla. El servidor no guarda nada sobre su conversación con un cliente concreto, aunque sí guarda el estado de los recursos (en Quizzy, las preguntas y los cuestionarios).

La calificación del cuestionario también es un buen ejemplo de esto. El cliente envía todas las respuestas en una sola petición y recibe la nota. Si las enviara una a una, el servidor tendría que recordar el progreso de cada cliente, es decir, mantener una sesión con estado. Para hacer algo así con REST, ese progreso tendría que modelarse como un recurso más (*p. ej.* `/attempts/{id}`).

Al no tener estado, el servidor es mucho más fácil de replicar y por tanto, el servicio escala mejor.

23.10.5. Validación y documentación

Sin un IDL compartido ni *stubs* generados, nada garantiza que una petición tenga la forma esperada. Cualquiera puede enviar cualquier cosa con `curl` o con una librería de HTTP como `requests`. Por eso el servidor tiene que validar todo lo que recibe.

Quizzy aplica tres niveles:

- Tipos y restricciones simples, *p. ej.* una pregunta debe tener entre 2 y 6 opciones. Pydantic las comprueba antes de ejecutar el manejador y, si fallan, FastAPI responde con un error 422 (*Unprocessable Content*).
- Restricciones entre campos, *p. ej.* el índice de la respuesta correcta debe ser menor que el número de opciones. Ninguno de los dos campos es incorrecto por separado. Pydantic también las comprueba, pero con un método escrito a mano, no con una declaración.
- Restricciones que dependen del estado, *p. ej.* que existan las preguntas que forman un cuestionario. Solo se pueden comprobar consultando la base de datos.

Los dos primeros niveles se declaran en el esquema `QuestionData` (Listado 23.39), con los argumentos de `Field()` y con el método `check_correct_option()`, respectivamente.

```
class QuestionData(BaseModel):
    text: str = Field(min_length=1)
    options: list[str] = Field(min_length=2, max_length=6)
    correct_option: int = Field(ge=0)

    @model_validator(mode='after')
    def check_correct_option(self) -> Self:
        if self.correct_option >= len(self.options):
```

```
            raise ValueError('correct_option must be the index of one of the options')
        return self
```

LISTADO 23.39: Esquema de los datos de una pregunta
/quizzy/server.py

Para describir una API REST se suele utilizar OpenAPI [72][5]. Un documento OpenAPI describe los recursos, sus operaciones y los esquemas de los datos, estos últimos con JSON Schema [73]. A partir de él, distintas herramientas generan documentación interactiva (como `Swagger UI`), *stubs* de cliente o de servidor y validadores.

Permite dos formas de trabajar:

- *design-first*: primero se escribe el documento OpenAPI y después el código, igual que con un IDL.
- *code-first*: el documento se genera a partir del código. Es lo que hace FastAPI, que lo publica en `/openapi.json`.

Un documento generado a partir del código solo recoge lo que está declarado. En Quizzy, el esquema de una pregunta limita el número de opciones, pero no dice nada de la relación entre la respuesta correcta y las opciones, porque esa regla solo está expresada en la implementación Python.

La gran diferencia con un IDL es que el documento OpenAPI es opcional. Ni el cliente ni el servidor lo necesitan para funcionar.

Sin embargo, describir así una API entra en conflicto con HATEOAS. El documento OpenAPI establece de antemano las rutas de todos los recursos (*p. ej.* `/quizzes/{id}`), y un cliente construido a partir de él depende de ellas, cuando en principio solo debería conocer una URI inicial y obtener las demás de los enlaces que incluye cada representación. Quizzy tampoco cumple esa restricción. Sus representaciones no incluyen enlaces y, aunque el servidor indica la ruta de cada recurso nuevo en la cabecera `Location`, `client.py` no la utiliza y compone las rutas a partir del identificador.

23.10.6. Errores y semántica de invocación

Con HTTP, los errores se notifican con el código de estado de la respuesta. Quizzy responde 404 (*Not Found*) si el recurso no existe y 422 si los datos no son válidos. El cuerpo puede llevar más detalles, y existe un formato estándar para eso en la RFC 9457 [74].

[5] Existen otros lenguajes con el mismo propósito, aunque su uso es mucho menos común

Con REST sobre HTTP (y por tanto sobre TCP), la semántica es *at-most-once*, igual que en gRPC. Un cliente solo debería reintentar automáticamente una petición si es idempotente [70].

23.11. Versionado

La interfaz de un servicio cambia con el tiempo. Aparecen operaciones nuevas, y otras necesitan un parámetro adicional o un tipo de mayor tamaño. En un programa local basta con recompilarlo todo, pero en un sistema distribuido los clientes y los servidores se compilan y despliegan por separado, a menudo por organizaciones distintas, así que durante un tiempo deben convivir versiones distintas de la misma interfaz. Un cambio es *compatible hacia atrás (backward compatible)* si los clientes antiguos pueden invocar servidores nuevos, y es *compatible hacia delante (forward compatible)* si los servidores antiguos admiten peticiones de clientes nuevos. Un cambio incompatible obliga al servidor a proporcionar ambas versiones a la vez y al cilente a indicar en las peticiones a qué versión se dirige.

Qué cambios son compatibles depende de dos decisiones de diseño del middleware: cómo identifica la petición la operación invocada y al modo en que serialización empaqueta los datos; si los etiqueta o simplemente los coloca uno detrás de otro. El caso más simple (como pico-rpc) es el caso más desfavorable en ambos aspectos. El identificador de cada operación es su posición en la interfa, así que si se inserta una operación delante de `factorial()`, cambian los identificadores arruinando la compatibilidad hacia atrás:

```
class Math(Interface):
    def square(n: uint8) -> uint64: ...               # 0 (nueva)
    def factorial(n: uint8) -> uint64: ...            # 1 (antes 0)
    def power(base: uint8, exp: uint8) -> uint64: ...  # 2 (antes 1)
```

Un cliente antiguo que invoca `factorial()` ejecuta en realidad `square()`, sin que ninguno de los dos extremos lo advierta.

ONC RPC incorpora la versión de la interfaz del propio protocolo. Cada petición incluye la versión como campo, junto a los números de programa y de procedimiento [75], y un mismo servidor puede ofrecer varias versiones de un programa. La contrapartida es que XDR no etiqueta los datos, así que cualquier cambio de tipos requiere una versión nueva. Es lo que necesitaría el ejemplo /`onc-rpc-factorial`, cuyo resultado (`unsigned long`) se codifica con 32 bits y llega truncado, sin ningún aviso, a partir de 13!. Para devolverlo como `unsigned hyper` (64 bits) basta con añadir un segundo bloque `version` al fichero `.x`:

```
program FACTORIAL_PROG {
   version FACTORIAL_VERS {
      unsigned long FACTORIAL(int n) = 1;
   } = 1;
   version FACTORIAL_VERS2 {
      unsigned hyper FACTORIAL(unsigned int n) = 1;
   } = 2;
} = 600000002;
```

`rpcgen` genera un *dispatcher* por versión, que el servidor registra por separado en `rpcbind`, y el número de versión que forma parte del nombre de las funciones permite que las implementaciones de ambas (`factorial_1_svc` y `factorial_2_svc`) coexistan en el mismo programa. Si el cliente solicita una versión que el servidor no ofrece, la respuesta de error (`PROG_MISMATCH`) indica la más baja y la más alta disponibles, y `clnt_create_vers()` utiliza esa información para conectar con la más alta del rango que admite el cliente:

```
rpcvers_t vers;   /* versión acordada: 1 o 2 */
clnt = clnt_create_vers(host, FACTORIAL_PROG, &vers, 1, 2, "udp");
```

Así ha evolucionado NFS (programa `100003`), y un mismo servidor puede atender a la vez a clientes de varias versiones.

gRPC no gestiona la compatibilidad con versiones de la interfaz, sino campo a campo. Cada campo de un mensaje se identifica por su número (§ 23.5.3), y el receptor ignora los números que no conoce, de modo que añadir un campo es compatible en ambos sentidos. Por eso tiene sentido que gRPC exija que cada operación reciba un mensaje, aunque contenga un único campo (`FactorialRequest`); añadir un parámetro equivale a añadir un campo a ese mensaje. Un campo se puede renombrar, porque el formato binario no incluye nombres, pero su número no se debe reutilizar nunca para otro campo, y la palabra clave `reserved` lo impide para los campos eliminados. Declarar el campo nuevo como `optional` permite al servidor saber si el cliente lo envió, ya que sin esa palabra un campo ausente se lee como 0:

```
message FactorialRequest {
  int32 value = 1;
  reserved 2;                    // campo eliminado
  optional uint64 modulus = 3;   // parámetro añadido
}
```

Las operaciones, en cambio, se identifican por su nombre completo, que forma la ruta de cada invocación (`/math.Math/factorial`), así que se pueden añadir, pero no renombrar. Para los cambios incompatibles, la convención es incluir la versión en el nombre del paquete (`package math.v2`). Cada versión se convierte así en un servicio distinto, con rutas distintas, y un mismo servidor puede ofrecer ambas.

JSON-RPC carece de interfaz y, por tanto, de cualquier mecanismo para versionarla. El campo `jsonrpc` indica la versión del protocolo, no la de la aplicación, y lo único que posibilita la evolución de la «interfaz» es el paso de argumentos por nombre, que permite añadir parámetros opcionales sin depender de su posición. Son los protocolos construidos sobre JSON-RPC los que resuelven el problema, cada uno a su manera. En LSP, cliente y servidor intercambian en la primera petición (`initialize`) la lista de funcionalidades opcionales que admite cada uno, sin ningún número de versión, y esa negociación vale para toda la sesión.

En el siguiente ejemplo, el cliente declara que puede mostrar en Markdown la información emergente sobre un símbolo *(hover)*, y el servidor, que es capaz de proporcionarla:

```
--> {"jsonrpc": "2.0", "id": 1, "method": "initialize", "params": {..., "capabilities":
      {"textDocument": {"hover": {"contentFormat": ["markdown", "plaintext"]}}}}}
<-- {"jsonrpc": "2.0", "id": 1, "result": {"capabilities": {"hoverProvider": true}}}
```

MCP identifica cada versión del protocolo con la fecha de su último cambio incompatible y, desde la revisión `2026-07-28`, la incluye en cada petición, junto con las funcionalidades del cliente, en lugar de negociarla al iniciar la sesión, para que el servidor no tenga que mantener estado entre peticiones:

```
--> {"jsonrpc": "2.0", "id": 7, "method": "tools/call", "params": {
      "name": "factorial", "arguments": {"n": 5},
      "_meta": {"io.modelcontextprotocol/protocolVersion": "2026-07-28",
                "io.modelcontextprotocol/clientCapabilities": {}}}}
```

Si el servidor no admite esa versión, responde con un error que enumera las que sí admite.

Java RMI identifica cada método por un *hash* de 64 bits calculado a partir de su nombre y de los tipos de sus parámetros y de su resultado. Añadir métodos a una interfaz remota es compatible, pero cualquier cambio en la firma de un método produce un *hash* distinto, y el servidor rechaza la invocación con una excepción (*unrecognized method hash*); la incompatibilidad se detecta. La peculiaridad es que la sobrecarga de métodos de Java permite conservar la versión antigua de una operación junto a la nueva. La interfaz *Hello* (Listado 23.19) puede incorporar una variante de `sayHello()` con un parámetro, y los clientes antiguos siguen invocando la original, porque para el middleware son métodos distintos; es extensible por sobrecarga:

```
public interface Hello extends Remote {
    String sayHello() throws RemoteException;
    String sayHello(String name) throws RemoteException;  // versión 2
}
```

Los objetos que se pasan por valor tienen su propia versión, el atributo `serialVersionUID` de su clase. Si el programador no lo declara, se calcula a partir de la estructura de la clase, y casi cualquier cambio impide deserializar los objetos de la versión anterior (`InvalidClassException`). Si lo declara y lo mantiene, la especificación de la serialización de Java establece qué cambios son compatibles, como añadir atributos, que toman su valor por omisión al leer un objeto antiguo:

```
public class Question implements Serializable {
    private static final long serialVersionUID = 1L;
    private String text;
    private String hint;   // añadido en la versión 2
}
```

En ICE cada petición indica la operación por su nombre y siempre lo hará de ese modo precisamente para que añadir operaciones nunca obligue a recompilar los clientes. Invocar una operación que el servidor desconoce produce un error explícito (`OperationNotExistException`). Los argumentos, en cambio, se serializan uno tras otro sin etiquetas, y nada en la petición identifica la firma. Si el servidor espera un parámetro que un cliente antiguo no envía, la invocación falla, pero un servidor antiguo que recibe un argumento de más lo ignora sin avisar. Para añadir parámetros de forma compatible, SLICE ofrece los *opcionales*, que llevan una etiqueta numérica con el mismo objetivo que el número de campo de Protocol Buffers y que también admiten los atributos de clases y excepciones:

```
interface Counter {
    long increment(optional(1) int step);
    idempotent long get();
};
```

Si el cliente no envía el argumento, el sirviente recibe el valor especial `Ice.Unset`:

```
def increment(self, step, current):
    self.value += 1 if step is Ice.Unset else step
    return self.value
```

Lo más peculiar de ICE son las *facetas (facets)*. Un objeto puede tener varias bajo la misma identidad, cada una con su propio sirviente y su propia interfaz, y cada petición indica a cuál de ellas se dirige (por omisión, a la faceta sin nombre). El servidor puede añadir una versión nueva de la interfaz como otra faceta de los objetos existentes sin alterar la que usan los clientes antiguos. El cliente nuevo la selecciona en el proxy, con la opción `-f` del proxy textual o con `ice_facet()`, y `checkedCast()` le permite comprobar si el objeto la tiene; si no la tiene, devuelve `None`:

```
# servidor
adapter.add(CounterI(), identity)
adapter.addFacet(CounterV2I(), identity, "v2")

# cliente
counter = Example.CounterV2Prx.checkedCast(base, "v2")
```

ICE advierte de que, a cambio, la elección de la interfaz se aplaza hasta la ejecución, con menos garantías de tipos *(type safety)* que si se hiciera en la compilación.

Cap'n Proto aplica el mismo enfoque Protocol Buffers, con una restricción que se deriva de su formato sin serialización. Los ordinales de los campos y de los métodos indican el orden en que se añadieron y deben ser consecutivos a partir de `@0`. El compilador rechaza cualquier hueco *(Ordinals must be sequential with no holes)* y asigna las posiciones en memoria siguiendo ese orden, de modo que un campo nuevo nunca desplaza a los existentes. Como no se admiten huecos, un campo no se puede eliminar, solo renombrar; el propio protocolo de Cap'n Proto conserva así algunos en desuso, como `obsoleteSave`.

Los parámetros de un método forman una estructura implícita, así que se pueden añadir al final con un valor por omisión, sin declarar un mensaje explícito como en GRPC:

```
interface TextProcessor {
  upper @0 (message :Text, prefix :Text = "") -> (result :Text);
}
```

Ccontempla incluso el caso de descubrir tarde que los elementos de una lista necesitan datos adicionales. Una lista de enteros (`List(UInt32)`) se puede convertir en una lista de estructuras cuyo primer campo (`@0`) es de ese mismo tipo, y cada versión lee correctamente los mensajes de la otra:

```
struct Stats {
  obsoleteName @0 :Text;      # en desuso, no se puede eliminar
  samples @1 :List(Sample);   # en la versión 1, List(UInt32)
  label @2 :Text;             # campo añadido
}

struct Sample {
  value @0 :UInt32;
  time @1 :UInt64;
}
```

En REST las operaciones no pueden cambiar, porque son las del protocolo (interfaz uniforme), y lo que evoluciona son los recursos y sus representaciones. Añadir un miembro a una representación JSON es compatible siempre que el receptor ignore los que no conoce, como hace Pydantic por omisión,

de modo que el servidor de Quizzy acepta sin más una pregunta con campos adicionales.

Para los cambios incompatibles, la versión se puede indicar en la URI (`/v2/quizzes`), en una cabecera propia o en el tipo de medio de la representación, que el cliente solicita mediante la negociación de contenido de HTTP. La primera opción es, con mucha diferencia, la más habitual, pero choca con el propio estilo, porque la URI identifica el recurso, y una petición es la misma independientemente de la versión de su representación. Fielding llegó a afirmar que un «v1» en la URI delata una API RPC sobre HTTP y no REST[6].

HTTP dispone de cabeceras específicas para anunciar en cada respuesta que un recurso está obsoleto (`Deprecation` [76]) y la fecha a partir de la cual dejará de estar disponible (`Sunset` [77]). Un cliente que solicitara la versión 1 de la representación de un cuestionario podría recibir una respuesta como esta, que la declara obsoleta desde el 1 de enero de 2026 (`Deprecation` expresa la fecha en segundos desde 1970) y anuncia su retirada el 30 de junio de 2027:

```
GET /quizzes/1 HTTP/1.1
Accept: application/vnd.quizzy.v1+json

HTTP/1.1 200 OK
Content-Type: application/vnd.quizzy.v1+json
Deprecation: @1767225600
Sunset: Wed, 30 Jun 2027 23:59:59 GMT
```

En cualquier caso, ningún middleware evita mantener dos versiones durante un tiempo cuando un cambio es incompatible. Lo que distingue a cada uno es dónde se indica qué versión invoca el cliente: un campo del mensaje en ONC RPC, el nombre del paquete en gRPC, la firma del método en Java RMI, la faceta en Ice, y la URI o el tipo de medio en REST.

23.12. Rendimiento

La invocación remota no es gratis. Esta sección cuantifica su coste comparando el tiempo de respuesta para una misma operación —`factorial(20)`— implementada con cada uno de los middlewares del capítulo. El tiempo se mide en el cliente, desde que invoca la operación hasta que recibe el resultado, así que incluye la serialización, la transmisión de los mensajes y el despachado en el servidor. El cálculo del factorial es despreciable frente a todo lo demás.

[6] `https://x.com/fielding/status/376835835670167552`

Las invocaciones son síncronas y secuenciales, sobre una conexión ya establecida. Cada dato es la media de 1000 invocaciones, descartando el 3 % más rápido y más lento para que unas pocas invocaciones anómalas, como las que sufren una retransmisión TCP, no distorsionen el resultado[7]. Se consideran tres destinos: el mismo proceso, otro proceso del mismo nodo y un proceso en otro nodo. En el mismo proceso se mide una llamada a función local convencional, es decir, la operación sin ninguna comunicación.

El lenguaje por sí solo ya influye en el resultado. Por eso comparamos implementaciones escritas en el mismo lenguaje (C++). En la comparación se incluye también cliente-servidor con sockets TCP sin middleware, que sirven de referencia; la columna «factor» indica cuántas veces más tarda cada middleware que los sockets. El Cuadro 23.6 compara cada mecanismo en C++, Java y Python, siempre en el mismo nodo. Java RMI aparece únicamente en este último, porque solo existe para Java.

	mismo nodo		**nodo remoto**	
Middleware	media	factor	media	factor
local	0,0075	-	-	-
sockets	15,4	1,00	35 406	1,0
ONC RPC	17,9	1,16	35 158	1,0
gRPC	79,0	5,13	35 819	1,0
Ice	27,7	1,80	35 220	1,0
Cap'n Proto	30,9	2,01	35 800	1,0
REST	37,3	2,42	35 455	1,0

CUADRO 23.5: Tiempo de respuesta de una invocación medido en el cliente (μs)

En el nodo de referencia, una llamada local en C++ tarda 0,0075 μs, mientras que la misma operación con sockets TCP hacia otro proceso del mismo nodo tarda 15,4 μs. Cruzar la frontera entre procesos multiplica el coste por 2052 antes incluso de elegir un middleware, debido a los cambios de contexto, las copias entre el espacio de usuario y el núcleo, y el paso por la pila de protocolos. Lo que añade cada middleware se suma a esa cifra.

ONC RPC es el que menos sobrecarga añade, porque su protocolo apenas envía algo más que el identificador de la operación y los argumentos. ICE añade el despachado en dos niveles (objeto y método) y una cabecera al-

[7]Antes de cada medida se descartan 5000 invocaciones. Esto se hace por el calentamiento *(warm-up)*. En el mismo nodo, cada medida se repite 10 veces, intercalada con las de los demás mecanismos, y se toma la mediana, porque el reparto de cliente y servidor entre los núcleos de la CPU varía de una ejecución a otra. La frecuencia de la CPU se mantiene fija en su máximo.

go mayor. Cap'n Proto queda ligeramente por encima de ICE. No serializa (§ 23.9), pero con un único entero como argumento esa ventaja no se aprecia. gRPC es, con diferencia, el más caro, porque cada invocación es una petición HTTP/2 completa, con su marco de cabeceras, su marco de datos y su control de flujo. A cambio, ofrece multiplexación sobre una misma conexión, *streaming* en ambos sentidos y la posibilidad de atravesar cualquier intermediario HTTP. REST, sobre HTTP/1.1 y con el resultado en JSON, queda entre Cap'n Proto y gRPC. Cada mensaje lleva cabeceras de texto que hay que componer e interpretar, pero no los marcos ni el control de flujo de HTTP/2.

Middleware	C++	Java	Python	cociente Python/C++
local	0,0075	0,0090	0,794	106
sockets	15,4	21,4	18,6	1,21
ONC RPC	17,9	-	-	-
gRPC	79,0	-	255,3	3,23
Java RMI	-	34,1	-	-
Ice	27,7	-	45,1	1,63
Cap'n Proto	30,9	-	165,5	5,35
REST	37,3	-	722,3	19,4

CUADRO 23.6: Tiempo de respuesta en el mismo nodo según el lenguaje (μs) y cociente entre el de Python y el de C++

El Cuadro 23.6 permite separar el coste del middleware del coste del lenguaje. En ICE, gRPC y Cap'n Proto, la versión en Python y la versión en C++ comparten middleware, protocolo y formato de serialización, y aun así el coste por invocación se multiplica por 1,63 en ICE, por 3,23 en gRPC y por 5,35 en Cap'n Proto. Buena parte de lo que se suele atribuir al middleware se debe en realidad al intérprete. En Cap'n Proto influye además que su soporte para Python no ofrece invocaciones síncronas. En REST la diferencia es aún mayor (19,4 veces), porque las dos versiones comparten el protocolo y el formato, pero no la implementación. Las bibliotecas de Python que utiliza Quizzy (§ 23.10.1) interpretan HTTP en el propio intérprete, mientras que las de ICE, gRPC y Cap'n Proto delegan el protocolo al *runtime* de C++.

Contra otro nodo, la escena cambia por completo. Todos los middlewares convergen alrededor de 35 406 μs, que es prácticamente el RTT al nodo remoto, y las diferencias entre ellos quedan por debajo del ruido. En cuanto hay una red de por medio, lo que determina el rendimiento es *cuántas*

invocaciones se hacen, no cuánto cuesta cada una. Un diseño que encadena muchas llamadas pequeñas (§ 23.6) cuesta RTT por cada una, así que la estrategia adecuada es evitar la espera —con invocaciones de solo petición, por lotes o asíncronas— más que elegir un middleware más ligero o un lenguaje más eficiente.

La media tampoco refleja el coste de la *primera* invocación, que incluye establecer la conexión y lo que cada middleware necesita antes de invocar, como la consulta al registro o `checkedCast()` en ICE. En Java incluye además la carga de las clases. En Java RMI, la primera invocación tarda 103 048 μs, frente a los 34,1 μs de media. Para un servicio que se ejecuta durante mucho tiempo es irrelevante, pero para un proceso que arranca, invoca una vez y termina, esa primera invocación es todo el coste.

A la vista de todas estas medidas, podemos concluir que:

- Una invocación remota es varios órdenes de magnitud más costosa que una local.
- La mayor parte de ese coste se debe a la comunicación entre procesos y sobre todo a la red, no al middleware.
- Cuanto mayor sea el coste computacional de la operación remota, más rentable resulta.
- El rendimiento depende principalmente del número de invocaciones. Cuantos menos mensajes y más grandes, mejor. La latencia de la red es el factor dominante, algo que ocurre a todos los niveles de la pila de protocolos.
- El lenguaje de implementación puede influir tanto como el middleware, que sigue siendo muy poco en relación al total.

Y ¿qué más?

La invocación remota sustituye el diseño de un protocolo de aplicación para cada servicio por una infraestructura genérica: el middleware. Genera los *stubs* a partir de una interfaz y se encarga de serializar, transportar y despachar cada invocación. Por debajo, sigue habiendo un protocolo de petición-respuesta, y las decisiones sobre el estilo de intercambio y las medidas de tolerancia a fallos determinan la semántica de invocación, es decir, qué garantías tiene el cliente cuando se pierde un mensaje.

Los middlewares docentes pico-rpc y pico-rmi muestran esas piezas en su forma más simple. ONC RPC, GRPC, Java RMI y ICE muestran cómo las resuelven los middlewares reales, que además se ocupan de localizar los servidores y de aprovechar el tiempo de espera con la invocación y el despachado asíncronos.

Cap'n Proto y REST cuestionan parte de ese planteamiento, cada uno por un camino distinto. Cap'n Proto mantiene el modelo de objetos, pero no intenta que una invocación remota se parezca a una local. Cada invocación devuelve una promesa, y cada referencia designa un objeto y concede a la vez el permiso para invocarlo. REST prescinde de IDL, compilador y *stubs*, y en lugar de que cada interfaz defina sus propias operaciones, aplica el mismo conjunto reducido a todos los recursos. JSON-RPC prescinde igualmente de ellos, aunque sin abandonar el modelo RPC; se limita a definir el formato de los mensajes, cada uno de los cuales indica el nombre de la operación invocada.

La petición-respuesta tampoco es la única base sobre la que se pueden construir abstracciones de comunicación de alto nivel. Las colas de mensajes y el modelo de actores, por ejemplo, resuelven necesidades similares con mensajes asíncronos en un solo sentido, sin intentar parecerse a una llamada a función, en la línea del modelo publicador-suscriptor (capítulo 16).